

Cybersecurity Attack & Defense Lifecycle

Institution	Yarmouk University
Course Name	MASAR Training
Environment	Controlled academic lab — Kali Linux / Tailscale / Splunk Enterprise
Submission Date	12 September 2026

TEAM MEMBERS AND ROLE OWNERSHIP

Role	Student	Primary responsibility
Student 1 — Architecture & Visibility	Yousef Moayad	VM1 target, vulnerable application, application architecture
Student 2 — Offensive / Red Team	Hadi Bakeer	Black-box testing, exploitation, proof of access, re-testing
Student 3 — Blue Team / IR	Mohammad Aiman Ali (ID 2023904141)	Splunk SIEM, timeline reconstruction, containment, detection engineering
Student 4 — Mitigation & Re-Exploitation	Mohammad Aiman Ali (ID 2023904141)	Git baseline, code remediation, redeployment, patch verification, and re-exploitation coordination

Abstract

This project simulated a complete attack-and-defense lifecycle against a deliberately vulnerable Urban Sports Management web application. Because the team members worked from separate locations and each Kali Linux virtual machine ran on a different computer, Tailscale was used as the private VPN overlay that connected the target (VM1), SIEM (VM2), and attacker (VM3). VM1 hosted the Flask-based application and Splunk Universal Forwarder, VM2 hosted Splunk Enterprise, and VM3 was the offensive workstation.

The Red Team assessment confirmed four security weaknesses: SQL injection in the login workflow, OS command injection leading to controlled remote code execution, stored cross-site scripting, and unrestricted file upload. The Blue Team reconstructed the incident from Apache telemetry, correlated upload requests with files on VM1, removed the attack-window artifacts, performed scoped persistence checks, and implemented Splunk searches for each vulnerability. The remediation phase then introduced parameterized SQL, strict IP validation with shell-free command execution, file-extension restrictions, and normal Jinja output escaping. Re-testing of the same or equivalent proof-of-concept inputs showed that the tested exploit impacts were no longer reproducible while the SIEM detections continued to fire on the re-test traffic.

Evidence discipline. HTTP response codes are treated as request/response evidence, not proof of successful exploitation. Exact POST-body payloads are not claimed from Apache access logs because the bodies were not recorded. Likewise, the absence of malicious entries in the inspected persistence locations is reported as “no obvious persistence identified,” not as an absolute guarantee of absence.

Contents

Section 1: Introduction	5
1.1 Project Background and Motivation	5
1.2 Objectives and Scope	5
1.3 Team Structure and Role Distribution	5
1.4 Lab Environment Summary	6
Section 2: Architecture & Infrastructure	7
2.1 Lab Design Overview	7
2.2 Virtual Machine Specifications	7
2.2.1 VM1 Target Machine	7
2.2.2 VM2 SIEM Machine	8
2.2.3 VM3 Attacker Machine	8
2.3 Vulnerable Web Application	9
2.3.1 Application Overview and Technology Stack	9
2.3.2 Vulnerability 1: File Upload	9
2.3.3 Vulnerability 2: OS Command Injection	10
2.3.4 Vulnerability 3: Cross-Site Scripting (XSS)	10
2.3.5 Vulnerability 4: SQL Injection	10
2.4 Log Forwarding Setup	10
2.4.1 Agent Configuration on VM1	10
2.4.2 Apache Log Ingestion	10
2.4.3 Zsh History Capture	11
2.5 SIEM Deployment on VM2	11
2.5.1 Installation and Configuration	11
2.5.2 Log Verification and Index Confirmation	13
2.6 Architecture Diagram	13
Section 3: Penetration Testing Report	14
3.1 Engagement Overview	14
3.1.1 Scope and Rules of Engagement	14
3.1.2 Methodology (Black Box)	14
3.1.3 Tools Used	14
3.2 Reconnaissance and Enumeration	14
3.3 Vulnerability Assessment	15
3.3.1 Vulnerability 1 — File Upload	15
3.3.2 Vulnerability 2 — OS Command Injection	16
3.3.3 Vulnerability 3 — Stored XSS	16
3.3.4 Vulnerability 4 — SQL Injection	17
3.4 Exploitation	18
3.4.1 Exploit Chain Walkthrough	18
3.4.2 Reverse Shell Delivery	18
3.4.3 Proof of Access	18
3.6 Risk Rating Summary	19
3.7 Recommendations	19
Section 4: Incident Response & Detection	20
4.1 Investigation Methodology	20
4.2 SIEM-Based Timeline Reconstruction	20
4.2.1 Apache Log Analysis – Malicious Requests Identified	21
4.2.2 Shell History Analysis – Commands Executed Post-Compromise	25
4.2.3 Correlated Attack Timeline	26
4.3 Containment and Remediation	27
4.3.1 SSH Access to VM1	27
4.3.2 Webshell / Artifact Identification and Deletion	28
4.3.3 Persistence Check and Clearance Confirmation	31
4.4 Detection Engineering	33
4.4.1 Suspicious File Upload Detection	33
4.4.2 XSS Vulnerability Detection	34
4.4.3 OS Command Injection Detection	35
4.4.4 SQL Injection Detection	36

4.5 Query Validation – Evidence of Firing Against Log Data	37
Section 5: Mitigation & Re-Exploitation	38
5.1 Version Control Setup	38
5.1.1 Git Repository Structure	38
5.1.2 Initial Commit Vulnerable Baseline	39
5.2 Code Remediation	39
5.2.1 Fix for Vulnerability 1 — File Upload	39
5.2.2 Fix for Vulnerability 2 — OS Command Injection	40
5.2.3 Fix for Vulnerability 3 — Stored XSS	41
5.2.4 Fix for Vulnerability 4 — SQL Injection	43
5.3 Patched Application Deployment	45
5.4 Re-Exploitation Testing	46
5.4.1 Re-test Results Against PT Report Findings	46
5.4.2 Confirmation of Patch Effectiveness	48
5.5 Detection Query Verification (By Student 3)	49
5.5.1 Re-run Exploit Attempt on Patched App	49
5.5.2 Confirmation That Student 3 Queries Fire on Re-test Logs	49
5.6 Git Commit History Summary	51
Section 6: Conclusion	53
6.1 Project Summary	53
6.2 Key Findings Across All Roles	53
6.3 Lessons Learned	53
6.4 Future Improvements	53
References	54

Section 1: Introduction

1.1 Project Background and Motivation

The project was designed to reproduce the practical lifecycle of a web-application security incident rather than treating offensive and defensive activities as isolated exercises. The team first created an intentionally vulnerable application, established centralized visibility, performed a controlled black-box assessment, reconstructed and contained the resulting activity, patched the vulnerable code, and repeated the tests to verify both prevention and continued detection.

A practical constraint shaped the architecture: team members were not on the same physical network, and each virtual machine ran on a different participant's computer. The team therefore used Tailscale to provide private IP connectivity between the three Kali Linux systems without exposing the lab directly to the public Internet.

1.2 Objectives and Scope

- Deploy and operate the Urban Sports Management application on VM1 with four intentional web vulnerabilities.
- Provide centralized logging from VM1 to Splunk Enterprise on VM2 before offensive testing begins.
- Perform authorized black-box reconnaissance and exploitation from VM3 without denial-of-service or destructive activity.
- Reconstruct the incident timeline, validate host impact, remove identified attack artifacts, and assess common persistence locations.
- Patch the four confirmed findings, redeploy the application, and repeat the proof-of-concept tests.
- Verify that the defensive searches still identify the post-patch re-test attempts.

1.3 Team Structure and Role Distribution

Table 1.1 – Actual Role Ownership Used in the Lab

Role	Assigned student	Contribution in the final lifecycle
Student 1 — Architecture & Visibility	Yousef Moayad	Built and operated VM1 and the vulnerable Urban Sports application.
Visibility / Student 3 SIEM	Mohammad Aiman Ali (ID 2023904141)	Operated Splunk Enterprise and the VM1→VM2 log pipeline; later performed incident response and detection engineering.
Student 2 — Offensive	Hadi Bakeer	Performed reconnaissance, exploitation, reverse-shell proof, evidence capture, and remediation re-testing.
Student 4 — Mitigation	Mohammad Aiman Ali (ID 2023904141)	Maintained the Git baseline, implemented the security fixes, redeployed the application, and coordinated patch verification.

The implementation responsibility differs slightly from the generic four-role template: Yousef Moayad owned VM1 and the vulnerable application, Hadi Bakeer performed the Red Team assessment, and Mohammad Aiman Ali owned the Splunk/incident-response work and also completed the Student 4 mitigation and re-exploitation responsibilities. The final report preserves the required section numbering while recording the actual work performed.

1.4 Lab Environment Summary

Table 1.2 – Lab Systems

System	Tailscale IP	Function	Key services / tools
--------	--------------	----------	----------------------

System	Tailscale IP	Function	Key services / tools
VM1 — Target	100.64.248.91	Application and evidence host	Kali Linux, Flask, SQLite, Apache/Gunicorn, Splunk Universal Forwarder, Tailscale
VM2 — SIEM	100.81.218.80	Central logging and investigation	Kali Linux, Splunk Enterprise, SSH administration, Tailscale
VM3 — Attacker	100.85.187.114	Controlled offensive workstation	Kali Linux, Nmap, FFUF, cURL, browser, Netcat, Tailscale

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ sudo tailscale status
[sudo] password for kali:
100.81.218.80   kali-1   drbarbari71@  linux   -
100.85.187.114 kali-2   hadibak2019@  linux   -
100.64.248.91  kali    Yo7sev@       linux   offline, last seen 12h ago
(kali㉿kali)-[~]
$
```

Figure 1.1 – Tailscale peer status showing the three private lab addresses. The screenshot records VM1 as offline at that later check, while the incident evidence separately confirms active connectivity during testing.

Section 2: Architecture & Infrastructure

PRIMARY CONTRIBUTORS: YOUSEF MOAYAD — VM1/APPLICATION; MOHAMMAD AIMAN ALI — SIEM/VISIBILITY

2.1 Lab Design Overview

The environment separated target, monitoring, and attacking responsibilities across three Kali Linux virtual machines. Tailscale created the private routed overlay between them. VM1 exposed the Urban Sports Management application to the lab, forwarded selected telemetry to VM2, and accepted administrative SSH from VM2. VM3 originated all black-box testing traffic. This separation allowed the team to correlate offensive actions with SIEM and host evidence without relying on a shared campus or home LAN.

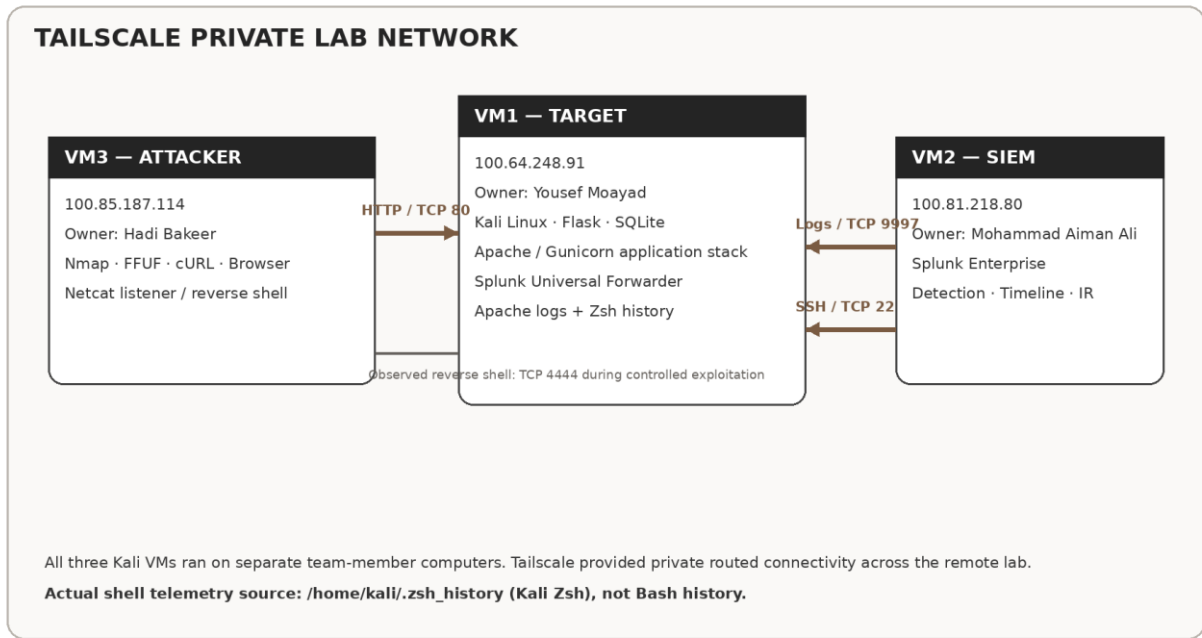


Figure 2.1 – Corrected final architecture showing Tailscale connectivity, application traffic, Splunk forwarding, SSH administration, and the actual Zsh history source.

2.2 Virtual Machine Specifications

The supplied evidence records operating system, service role, private IP addressing, application paths, and security tooling. CPU, memory, and virtual-disk allocations were not preserved in the screenshots and are therefore not invented in this report.

2.2.1 VM1 Target Machine

Property	Verified configuration
Operating system	Kali Linux
Tailscale IP	100.64.248.91
Application path	/home/kali/vulnerable-app
Application components	Python/Flask, Jinja templates, SQLite database, uploads directory
Front-end / service stack	Apache and the vulnerable-app service were active; offensive HTTP responses identified Gunicorn
Telemetry agent	Splunk Universal Forwarder

```

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo systemctl is-active tailscaled
[sudo] password for kali:
active

(kali@kali)-[~]
$ sudo systemctl is-active apache2
active

(kali@kali)-[~]
$ sudo systemctl is-active vulnerable-app
active

(kali@kali)-[~]
$ ls -la ~/vulnerable-app
total 44
drwxrwxr-x 7 kali kali 4096 Sep  5 17:03 .
drwx----- 20 kali kali 4096 Sep 11 08:47 ..
-rw-rw-r-- 1 kali kali 4004 Sep  5 16:53 app.py
-rw-r--r-- 1 kali kali 12288 Sep  5 16:56 database.db
drwxrwxr-x 2 kali kali 4096 Sep  5 17:03 __pycache__
drwxrwxr-x 2 kali kali 4096 Sep  5 16:56 static
drwxrwxr-x 2 kali kali 4096 Sep  5 18:34 templates
drwxrwxr-x 2 kali kali 4096 Sep  7 13:34 uploads
drwxrwxr-x 5 kali kali 4096 Sep  5 16:51 venv

```

Figure 2.2 – VM1 service verification: *tailscaled*, *apache2*, and *vulnerable-app* are active, followed by a listing of the application directory.

2.2.2 VM2 SIEM Machine

Property	Verified configuration
Operating system	Kali Linux
Tailscale IP	100.81.218.80
SIEM	Splunk Enterprise
Receiver	TCP 9997
Index	vulnerable_app
Use	Log ingestion, searches, timeline reconstruction, detection validation, SSH administration

2.2.3 VM3 Attacker Machine

Property	Verified configuration
Operating system	Kali Linux
Tailscale IP	100.85.187.114
Primary tools	Nmap, FFUF, cURL, Web Browser/DevTools, Netcat
Scope	Urban Sports Management web application on VM1
Testing model	Authorized black-box security assessment

2.3 Vulnerable Web Application

2.3.1 Application Overview and Technology Stack

Urban Sports Management is a Flask-based management application with authenticated functions for teams, players, matches, documents, announcements, and a system-status connectivity check. The evidence shows a Python application file (app.py), Jinja templates, static assets, an uploads directory, and database.db. The offensive assessment identified Gunicorn in the HTTP server response, and VM1 service evidence showed Apache active in front of the application environment.

```

kali@kali: ~/vulnerable-app
Session Actions Edit View Help
(kali@kali)-[~]
$ cd ~/vulnerable-app
tree -L 2
.
├── app.py
├── app.py.backup
├── database.db
├── database.db.backup
├── database.db.before-role-fix
├── __pycache__
│   └── app.cpython-314.pyc
├── static
│   ├── style.css
│   └── static.backup
│       └── style.css
├── templates
│   ├── base.html
│   ├── command.html
│   ├── dashboard.html
│   ├── diagnostics.html
│   ├── documents.html
│   ├── index.html
│   ├── login.html
│   ├── matches.html
│   ├── players.html
│   ├── teams.html
│   ├── upload.html
│   └── xss.html
├── templates.backup
│   ├── command.html
│   ├── index.html
│   ├── login.html
│   ├── upload.html
│   └── xss.html
├── uploads
│   ├── lab-test.txt
│   ├── poc.php
│   ├── shelll.php
│   └── test.txt
└── venv
    ├── bin
    ├── include
    ├── lib
    ├── lib64 → lib
    └── pyvenv.cfg

12 directories, 30 files

```

Figure 2.3 – VM1 application directory tree showing app.py, database.db, templates, static assets, uploads, and the Python virtual environment.

2.3.2 Vulnerability 1: File Upload

The original Documents function saved uploaded files using the supplied filename without an effective allowlist. The controlled test therefore accepted upload-test-2026.php. This demonstrated insufficient file-type validation, but the assessment did not demonstrate direct PHP execution and did not classify the file as a confirmed webshell.

2.3.3 Vulnerability 2: OS Command Injection

The Diagnostics / System Status feature accepted a host value and passed attacker-controlled content to an operating-system command. The Red Team demonstrated command execution as the kali user and then established a controlled reverse shell. This was the highest-impact code-execution path in the application.

2.3.4 Vulnerability 3: Cross-Site Scripting (XSS)

The Community Announcements feature rendered stored announcement content without safe output encoding. A controlled script payload executed when the stored announcement was displayed and executed again after refresh, supporting classification as Stored/Persistent XSS.

2.3.5 Vulnerability 4: SQL Injection

The login workflow built SQL from username and password input. A Boolean-true injection condition bypassed authentication and created an Administrator application session. A separate Players search parameter produced SQL-related behavior and was later used for SIEM URI-level SQL-injection detection testing.

2.4 Log Forwarding Setup

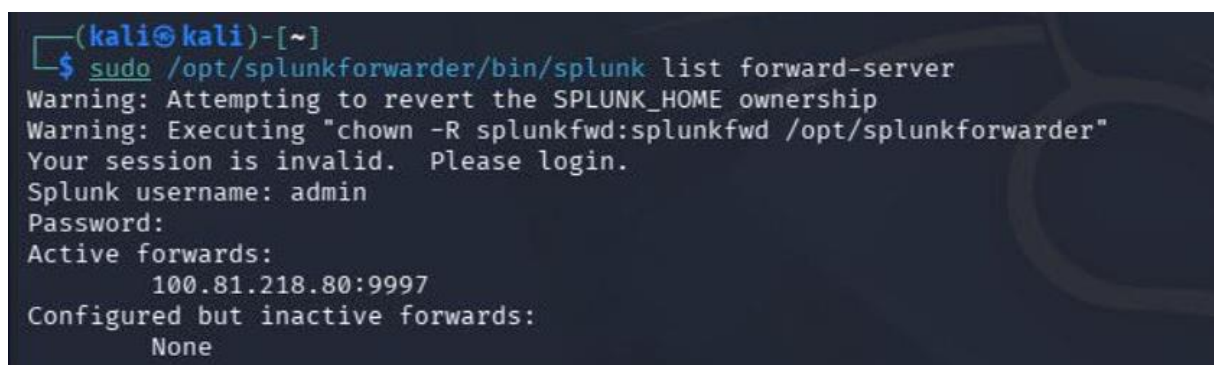
2.4.1 Agent Configuration on VM1

Splunk Universal Forwarder ran on VM1 and forwarded to Splunk Enterprise on VM2 at 100.81.218.80:9997. The forward-server status screenshot explicitly lists this destination as active. The evidence proves the effective destination, monitored sources, index, and sourcetypes; it does not preserve the exact original command sequence used during installation.

```
Effective forwarding destination
VM2 receiver: 100.81.218.80:9997

Monitored sources
/var/log/apache2/vulnerable-app-access.log -> index=vulnerable_app, sourcetype=vulnerable_app:apache
/home/kali/.zsh_history -> index=vulnerable_app,
sourcetype=vulnerable_app:zsh_history
```

SIEM EVIDENCE — UNIVERSAL FORWARDER



```
(kali@kali)-[~]
$ sudo /opt/splunkforwarder/bin/splunk list forward-server
Warning: Attempting to revert the SPLUNK_HOME ownership
Warning: Executing "chown -R splunkfwd:splunkfwd /opt/splunkforwarder"
Your session is invalid. Please login.
Splunk username: admin
Password:
Active forwards:
    100.81.218.80:9997
Configured but inactive forwards:
    None
```

Figure 2.4 – Universal Forwarder Connected to VM2

Evidence interpretation. The forward-server status lists 100.81.218.80:9997 under Active forwards and shows no inactive forward. This verifies the effective VM1-to-VM2 forwarding path.

2.4.2 Apache Log Ingestion

The Apache access log was ingested under sourcetype vulnerable_app:apache. It provided timestamps, source addresses, request methods, URIs, status codes, and user-agent strings. Standard access logging did not record POST bodies, which is important when interpreting the XSS and command-injection events.

SIEM EVIDENCE — APACHE COLLECTION

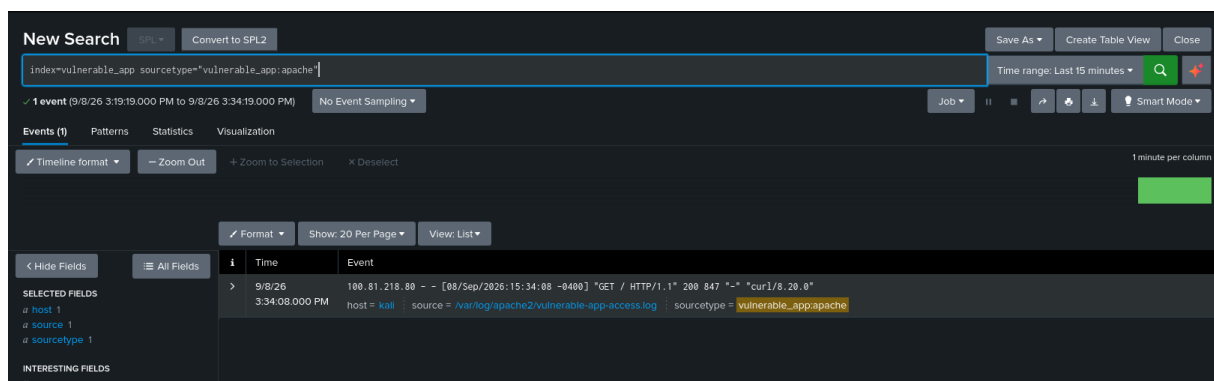


Figure 2.5 – Pre-Attack Apache Ingestion Baseline

Evidence interpretation. A benign event is visible in Splunk under `index=vulnerable_app` and `sourcetype=vulnerable_app:apache`. The event establishes that Apache access telemetry was searchable before offensive activity began.

2.4.3 Zsh History Capture

Although the generic project template refers to Bash history, Kali used Zsh. The monitored shell-history source was therefore `/home/kali/.zsh_history` with `sourcetype vulnerable_app:zsh_history`. Benign pre-attack commands were visible, proving the source worked before the incident. Commands executed by the web process were not written to this interactive shell history.

SIEM EVIDENCE — ZSH COLLECTION

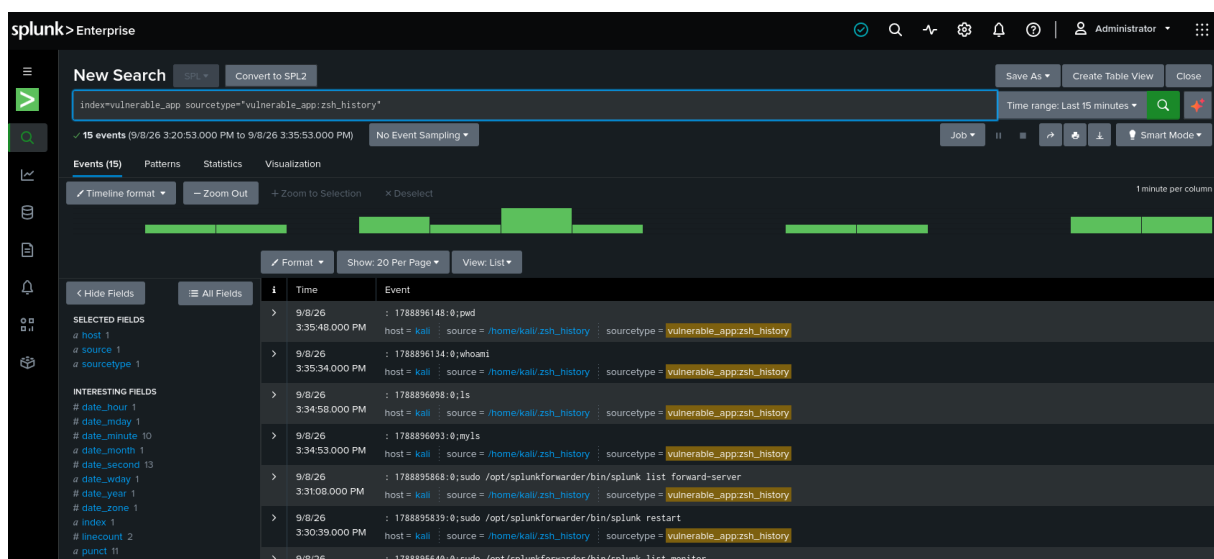


Figure 2.6 – Pre-Attack Zsh History Ingestion Baseline

Evidence interpretation. Benign commands such as `pwd` and `whoami` are visible under `vulnerable_app:zsh_history`. This confirms that Kali Zsh history, rather than Bash history, was successfully forwarded before the attack.

2.5 SIEM Deployment on VM2

2.5.1 Installation and Configuration

Splunk Enterprise on VM2 served as the central SIEM. The receiver accepted Universal Forwarder traffic on TCP 9997, and searches were performed against the `vulnerable_app` index. The deployment was operational before the attack window, which allowed the investigation to establish a reliable chronology after testing began.

Verified SIEM settings

Splunk host: VM2 / 100.81.218.80

Receiving port: TCP 9997

Index: vulnerable_app

Apache sourcetype: vulnerable_app:apache

Shell-history sourcetype: vulnerable_app:zsh_history

SIEM EVIDENCE — SPLUNK RECEIVER

```
(kali@kali)-[~/Masar]
$ ss -lntp | grep 9997
LISTEN 0      128          0.0.0.0:9997  0.0.0.0:*
```

Figure 2.7 – Splunk Enterprise Receiver Listening on TCP 9997

Evidence interpretation. The VM2 socket listing shows a listener on 0.0.0.0:9997. This is the receiving service used for Splunk Universal Forwarder traffic from VM1.

2.5.2 Log Verification and Index Confirmation

The team verified both network reachability and ingestion before offensive testing. Tailscale status and ping confirmed connectivity to VM1, while Splunk searches returned both an Apache access event and benign Zsh history. This baseline matters because missing attack-window shell events can then be treated as a source-coverage limitation rather than as a setup failure.

SIEM EVIDENCE — NETWORK REACHABILITY

```
(kali@kali)-[~/Masar]
$ tailscale status
100.81.218.80   kali-1   drbarbari71@  linux -
100.85.187.114 kali-2   hadibak2019@  linux -
100.64.248.91  kali    Yo7sev@       linux -

(kali@kali)-[~/Masar]
$ ping 100.64.248.91
PING 100.64.248.91 (100.64.248.91) 56(84) bytes of data.
64 bytes from 100.64.248.91: icmp_seq=1 ttl=64 time=1274 ms
64 bytes from 100.64.248.91: icmp_seq=2 ttl=64 time=440 ms
64 bytes from 100.64.248.91: icmp_seq=3 ttl=64 time=232 ms
^C
— 100.64.248.91 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2021ms
rtt min/avg/max/mdev = 232.127/648.433/1273.511/450.043 ms, pipe 2
```

Figure 2.8 – Tailscale Connectivity and Reachability to VM1

Evidence interpretation. Tailscale status lists the three private lab peers and the ping to 100.64.248.91 succeeds with no packet loss. This proves that the remote lab had routed connectivity before the security test.

2.6 Architecture Diagram

Figure 2.1 is the final corrected architecture diagram used by this report. It replaces the generic project schematic where “Bash history” was shown, because the actual monitored shell source was Zsh. The diagram also records the Tailscale layer that was essential to the remote-team design.

Section 3: Penetration Testing Report

STUDENT 2 — HADI BAKEER / OFFENSIVE SECURITY

3.1 Engagement Overview

3.1.1 Scope and Rules of Engagement

Testing was restricted to the modified Urban Sports Management application at 100.64.248.91 within the authorized Tailscale lab. The attacker system was 100.85.187.114. No denial-of-service or destructive actions were performed; proof-of-concept activity was limited to demonstrating security impact and collecting evidence.

3.1.2 Methodology (Black Box)

- Reconnaissance — identify reachable services and basic application behavior.
- Enumeration — discover ports, routes, and protected application functions.
- Vulnerability assessment — test authentication, upload, command execution, XSS, and injection points.
- Exploitation — validate confirmed weaknesses with controlled proof-of-concept inputs.
- Post-exploitation — establish a controlled reverse shell after RCE was confirmed.
- Reporting and handover — document evidence, risk, and remediation requirements.

3.1.3 Tools Used

Tool	Purpose
Nmap	TCP port and service enumeration
FFUF	Web route and endpoint discovery
cURL	HTTP request/response validation
Web browser / DevTools	Application interaction and evidence
Netcat	Controlled reverse-shell listener
Tailscale	Private connectivity to the authorized lab

3.2 Reconnaissance and Enumeration

Initial requests identified the application on TCP/80 and a full TCP scan also identified TCP/46875. Service probing showed the latter as a Go net/http response associated with the Tailscale environment rather than the Urban Sports application, so it was documented but not treated as an application vulnerability. FFUF then discovered routes including /login, /health, /diagnostics, /documents, /logout, /matches, /players, and /server-status.

```
(kali@kali)-[~/CyberSecurity/Masar]
$ curl -I --max-time 10 http://100.64.248.91
HTTP/1.1 302 FOUND
Date: Fri, 11 Sep 2026 14:22:42 GMT
Server: unicorn
Content-Type: text/html; charset=utf-8
Content-Length: 199
Location: /login
Vary: Cookie
```

Figure 3.1 – HTTP response from the target showing a redirect to /login and the Unicorn application response.

```
(kali@kali)-[~/CyberSecurity/Masar]
$ nmap -sS -sV -pN 100.64.248.91
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-11 10:28 -0400
Nmap scan report for kali.taile0c23f.ts.net (100.64.248.91)
Host is up (0.039s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Gunicorn

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.81 seconds
```

Figure 3.2 – Initial service enumeration identifying HTTP on TCP/80.

```
(kali@kali)-[~/CyberSecurity/Masar]
$ ffuf -u http://100.64.248.91/FUZZ \
-w /usr/share/wordlists/dirb/common.txt \
-mc 200,204,301,302,307,308,401,403 \
-fc 404 \
-t 20

v2.1.0-dev

:: Method      : GET
:: URL         : http://100.64.248.91/FUZZ
:: Wordlist    : FUZZ: /usr/share/wordlists/dirb/common.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 20
:: Matcher    : Response status: 200,204,301,302,307,308,401,403
:: Filter     : Response status: 404

diagnostics [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 4083ms]
documents   [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 44ms]
health      [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 55ms]
logout      [Status: 200, Size: 56, Words: 3, Lines: 2, Duration: 71ms]
matches     [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 40ms]
login       [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 67ms]
players     [Status: 200, Size: 1517, Words: 534, Lines: 83, Duration: 344ms]
server-status [Status: 302, Size: 199, Words: 18, Lines: 6, Duration: 82ms]
server-status [Status: 403, Size: 318, Words: 21, Lines: 10, Duration: 81ms]
:: Progress: [4614/4614] :: Job [1/1] :: 204 req/sec :: Duration: [0:00:19] :: Errors: 0 ::
```

Figure 3.3 – FFUF route discovery against the modified Urban Sports Management application.

3.3 Vulnerability Assessment

3.3.1 Vulnerability 1 — File Upload

The authenticated Documents function accepted a benign text file and also accepted upload-test-2026.php containing harmless PHP test code. The success message proves that the upload control did not adequately restrict file type. A request to the guessed public path returned HTTP 404, so the assessment records file acceptance but does not claim PHP execution or file-upload-to-RCE.

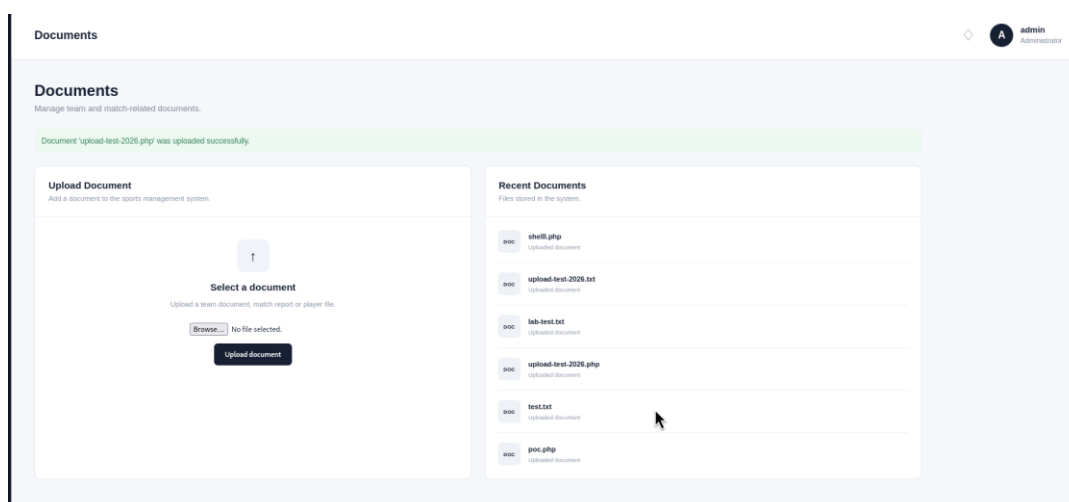


Figure 3.4 – Application confirmation that upload-test-2026.php was accepted by the Documents function.

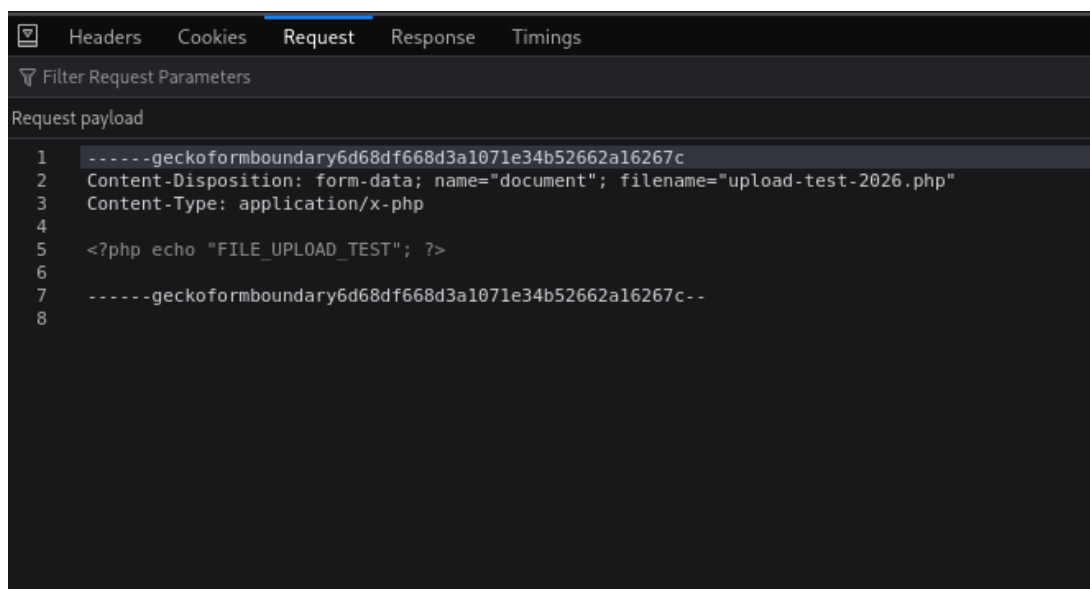


Figure 3.5 – Multipart upload request showing the PHP filename and benign FILE_UPLOAD_TEST content.

3.3.2 Vulnerability 2 — OS Command Injection

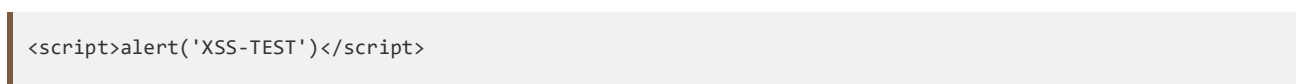
The System Status connectivity check accepted attacker-controlled input that was interpreted by a shell. A controlled test using 127.0.0.1; /usr/bin/id returned uid=1000(kali), and additional validation returned kali as the current user and /home/kali/vulnerable-app as the working directory. This directly demonstrated command execution in the application context.



Figure 3.6 – Command-injection proof returning uid=1000(kali).

3.3.3 Vulnerability 3 — Stored XSS

The Community Announcements feature stored attacker-supplied HTML/JavaScript. Publishing the controlled payload below caused a JavaScript alert to execute. The alert appeared again after page refresh without resubmission, supporting a Stored/Persistent XSS classification.



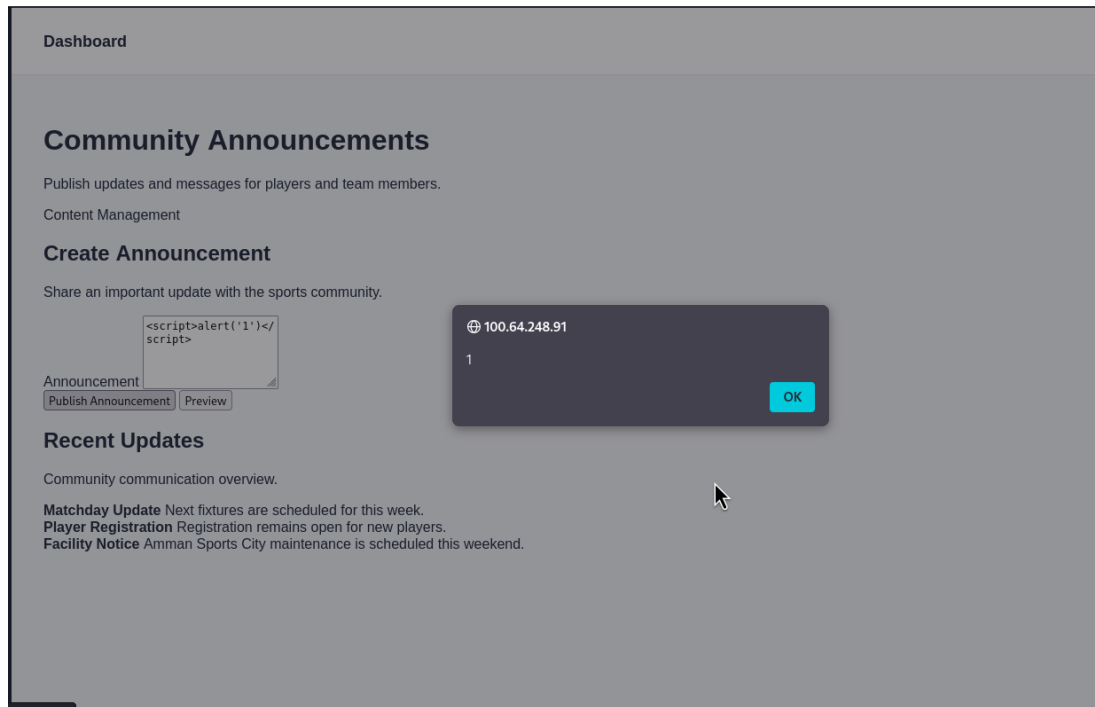


Figure 3.7 – JavaScript alert triggered after the malicious announcement content was stored and rendered.

3.3.4 Vulnerability 4 — SQL Injection

The login function was vulnerable to SQL injection. A Boolean-true username condition with password test bypassed authentication and produced an Administrator session, while a Boolean-false condition returned the normal authentication failure response. The result is stronger than an HTTP status alone because the evidence shows access to the Administrator dashboard.

```
Username: ' OR 1=1 --
Password: test
```

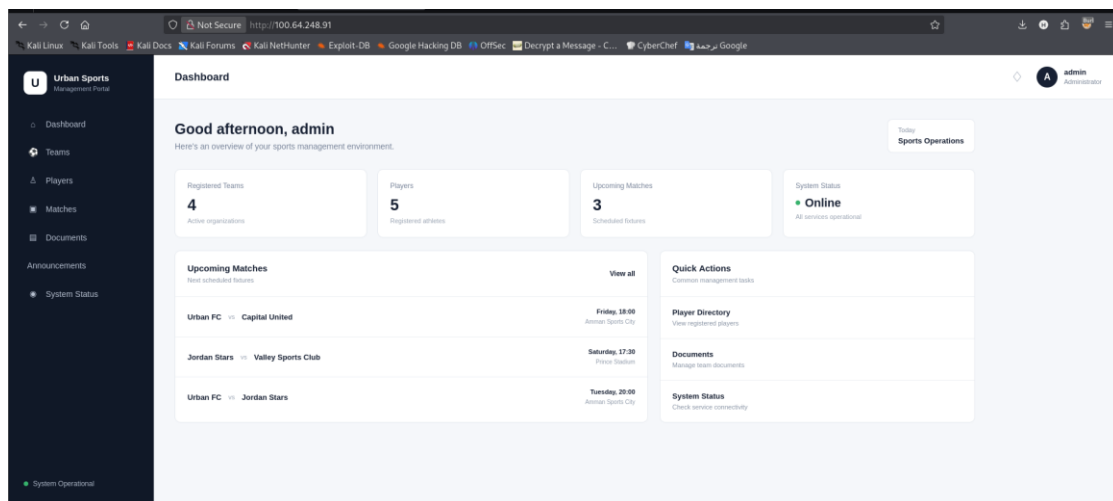


Figure 3.8 – Administrator dashboard reached after the SQL-injection authentication bypass.

3.4 Exploitation

3.4.1 Exploit Chain Walkthrough

Step	Observed chain
1	SQL injection in /login
2	Authentication bypass
3	Administrator application session
4	Access to protected Diagnostics
5	OS command injection
6	Remote code execution
7	Reverse-shell connection
8	Interactive shell as kali

3.4.2 Reverse Shell Delivery

After command execution was confirmed, the Red Team started a controlled Netcat listener on VM3 and triggered an outbound connection from VM1. The connection was received from 100.64.248.91 to 100.85.187.114 on TCP/4444. This evidence confirms the RCE impact; it is distinct from the later Blue Team network snapshot, where no active reverse-shell connection remained.

```
/usr/bin/nc -lvnp 4444
```

```
(kali@kali)-[~/CyberSecurity/Masar]
$ /usr/bin/nc -lvnp 4444
listening on [any] 4444 ...
connect to [100.85.187.114] from (UNKNOWN) [100.64.248.91] 46224
Command 'dircolors' is available in the following places
* /bin/dircolors
* /usr/bin/dircolors
The command could not be located because '/bin:/usr/bin' is not included in the PATH environment variable.
dircolors: command not found
kali@kali:~/vulnerable-app$
```

Figure 3.9 – Netcat listener receiving the controlled reverse-shell connection from VM1.

3.4.3 Proof of Access

The interactive shell was validated using non-destructive host-context commands. The output identified the current user as kali, the process identity as uid=1000(kali), the working directory as /home/kali/vulnerable-app, and the hostname as kali.

```
kali@kali:~/vulnerable-app$ /usr/bin/whoami
kali
kali@kali:~/vulnerable-app$ /usr/bin/id
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(netdev),102(scanner),110(wireshark),119(kaboxer),966(vboxsf),982(bluez),999(lpad
bin)
kali@kali:~/vulnerable-app$ /bin/pwd
/home/kali/vulnerable-app
kali@kali:~/vulnerable-app$ /bin/hostname
kali
kali@kali:~/vulnerable-app$
```

Figure 3.10 – Interactive shell proof of access and host-context validation.

Table 3.1 – Reverse-Shell Validation

Command	Result
/usr/bin/whoami	kali

Command	Result
/usr/bin/id	uid=1000(kali)
/bin/pwd	/home/kali/vulnerable-app
/bin/hostname	kali

3.6 Risk Rating Summary

Table 3.2 – Risk Rating Summary

Finding	Likelihood	Impact	Severity	Evidence basis
Login SQL Injection	High	Critical	Critical	Unauthenticated path to an Administrator session was demonstrated.
OS Command Injection / RCE	High	Critical	Critical	Arbitrary command execution and an interactive shell were demonstrated.
Unrestricted File Upload	High	High	High	A PHP file was accepted; execution was not demonstrated.
Stored XSS	Medium	Medium	Medium	Stored script execution persisted across refresh.

3.7 Recommendations

- SQL Injection — replace dynamic SQL construction with parameterized queries and keep authentication decisions server-side.
- OS Command Injection — validate IP/host input strictly and execute utilities without invoking a shell.
- File Upload — implement an allowlist, enforce server-side size/type checks, rename stored files where appropriate, and prevent script execution in upload locations.
- Stored XSS — rely on context-aware template escaping and avoid unsafe rendering of untrusted content.
- Monitoring — keep centralized web telemetry and add application-level security events and process-execution telemetry for better command visibility.

Section 4: Incident Response & Detection

STUDENT 3 — MOHAMMAD AIMAN ALI / UNIVERSITY ID 2023904141

4.1 Investigation Methodology

The investigation followed a SIEM-first incident-response workflow. Splunk Enterprise on VM2 was used to preserve chronology before host cleanup. The official original attack window was 10:29:15–11:38:23 EDT on 11 September 2026, and the primary attacker source was VM3 at 100.85.187.114. After web-log triage, VM1 was inspected over authorized SSH to validate artifacts, processes, connections, and common persistence mechanisms.

Table 4.1 – Investigation Environment

Component	Role / verified value
VM1	Target — 100.64.248.91
VM2	Splunk Enterprise SIEM — 100.81.218.80
VM3	Attacker — 100.85.187.114
Log transport	Universal Forwarder → VM2 over TCP 9997
Index	vulnerable_app
Apache sourcetype	vulnerable_app:apache
Shell sourcetype	vulnerable_app:zsh_history

INCIDENT SCOPING EVIDENCE

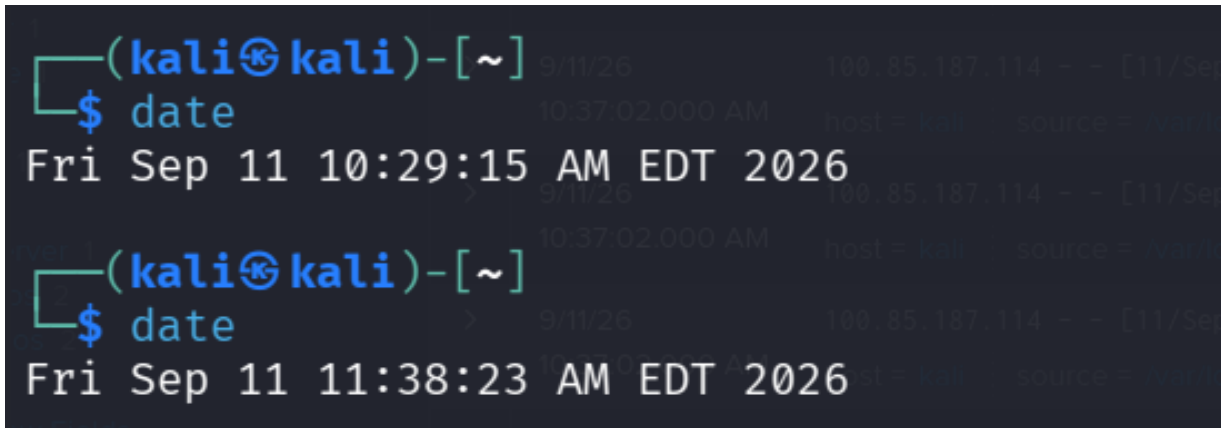


Figure 4.1 – Official Attack Window Recorded on VM3

Evidence interpretation. The start and stop commands record 10:29:15 and 11:38:23 EDT on 11 September 2026. These timestamps define the official investigation window used in the SIEM searches.

4.2 SIEM-Based Timeline Reconstruction

The sourcetype summary returned 47,780 Apache events in the official window and no Zsh-history row. A separate source-based summary identified 47,720 requests from VM3. The difference between the total index result and attacker-source result is expected because the broader window also contains supporting or non-VM3 traffic.

SIEM ATTACK-WINDOW SCOPE

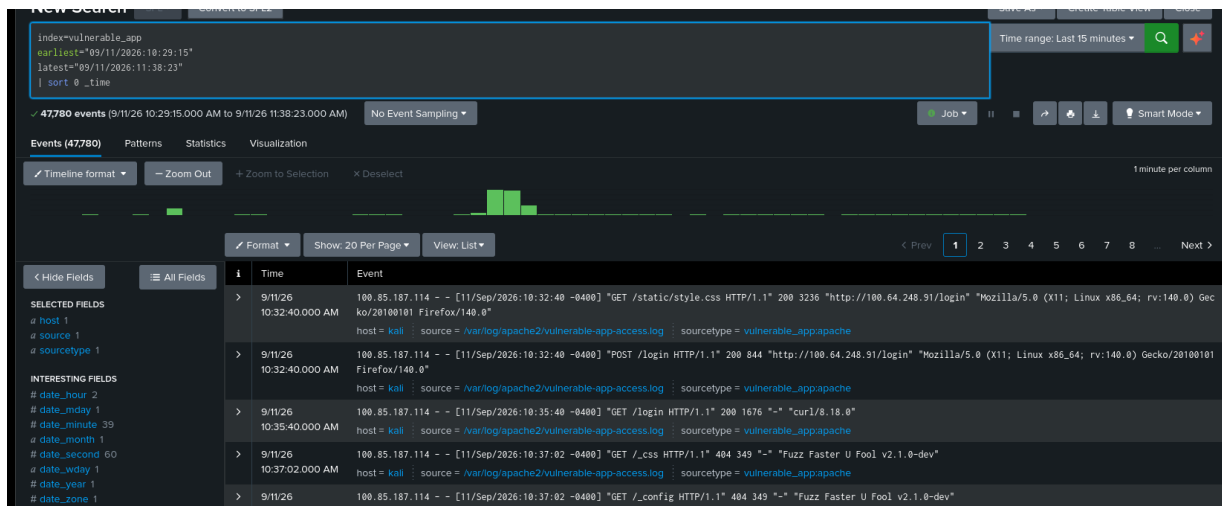


Figure 4.2 – Splunk Attack-Window Event Scope

Evidence interpretation. The vulnerable_app search is bounded to the official attack window and returns the Apache events generated during that period. The attacker source 100.85.187.114 is visible in the event data.

4.2.1 Apache Log Analysis – Malicious Requests Identified

Apache telemetry showed a clear sequence: login activity, large-scale FFUF enumeration, access to protected application functions, file uploads, Diagnostics requests, XSS requests, and SQL-injection-style probes. FFUF generated 47,621 requests against 43,765 unique URIs between 10:37:02 and 10:58:22 EDT. Of these, 47,602 returned 404, four returned 200, and thirteen returned 302, which is consistent with automated route enumeration.

SQL-injection-style inputs were visible where they appeared in the URI, including a quote, Boolean conditions such as ' OR 1=1 -- and ' OR 1=2 --, and a UNION SELECT probe against /players?search=. In contrast, Apache did not record POST bodies, so the exact XSS and command-injection strings used in form submissions were not recoverable from these access logs.

APACHE INVESTIGATION EVIDENCE

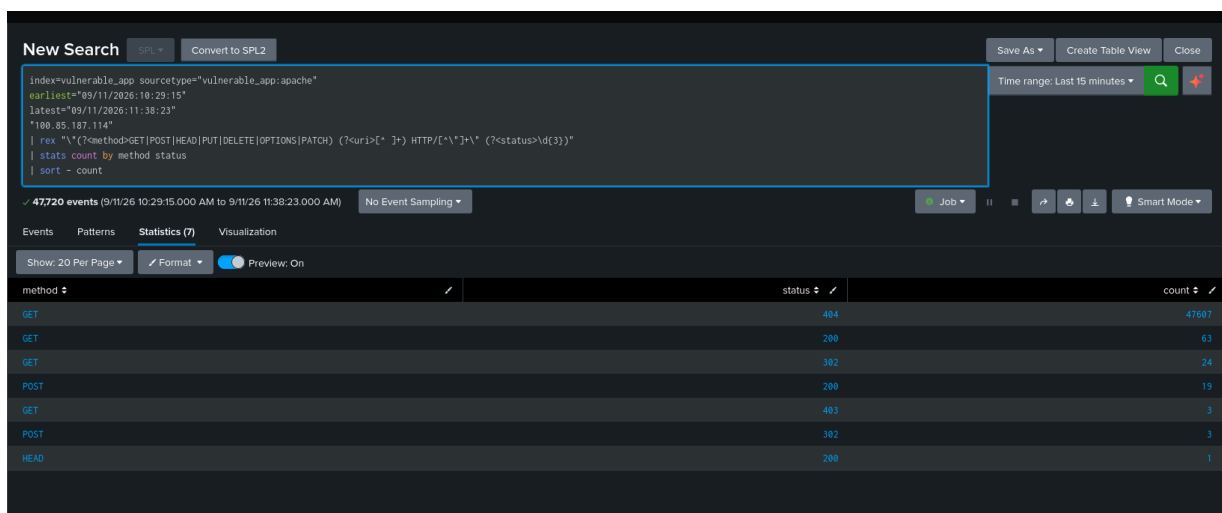


Figure 4.3 – HTTP Method and Status Distribution

Evidence interpretation. The statistics search groups the attack-window traffic by HTTP method and response status. It provides an overview of GET, POST, HEAD and response-code behavior before individual endpoints are examined.

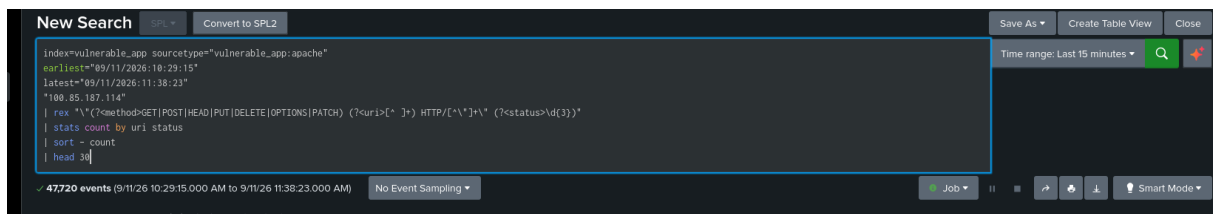


Figure 4.4 – Top-URI Investigation Query

Evidence interpretation. This search extracts the URI field and counts requests by URI and status for VM3. It is used to identify the most frequently requested paths during reconnaissance and exploitation.

uri	status	count
/static/style.css	200	32
/login	200	12
/diagnostics	200	7
/documents	200	7
/players	302	6
/health	200	5
/diagnostics	302	4
/documents	302	4
/xss	200	4
/login	302	3
/logout	302	3
/matches	302	3
/players	200	3
/server-status	403	3
/	200	2
/	302	2
/.bash_history	404	2
/.bashrc	404	2
/.cache	404	2
/.conf	404	2

Figure 4.5 – Top Requested URIs — First Result Page

Evidence interpretation. The first result page shows application endpoints and high-volume paths observed from VM3. The list helps separate normal application routes from automated probing.

uri	status	count
/.history	404	2
/.hta	404	2
/.htaccess	404	2
/.htpasswd	404	2
/.mysql_history	404	2
/.ssh	404	2
/.svn	404	2
/.swf	404	2
/.web	404	2
/.	404	2

Figure 4.6 – Top Requested URIs — Continuation

Evidence interpretation. The continuation includes dot-file and history-style paths characteristic of automated content discovery. These entries support the conclusion that directory fuzzing was performed.

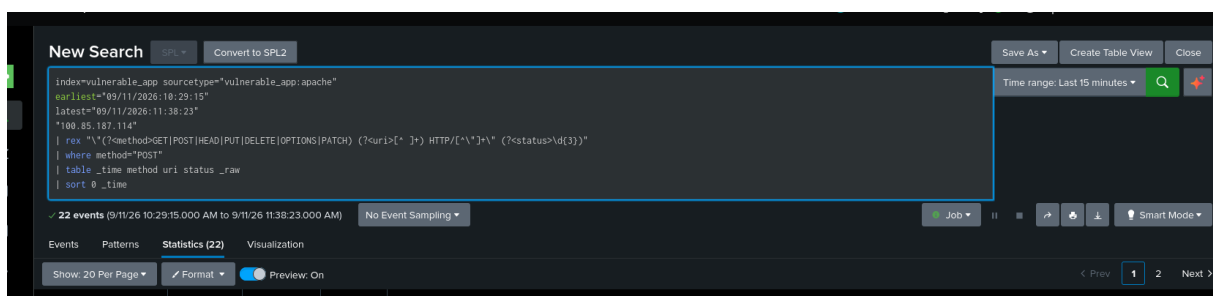


Figure 4.7 – POST-Only Attack-Window Query

Evidence interpretation. The query filters the attacker traffic to POST requests and displays time, method, URI, status and raw event data. This isolates state-changing interactions with login, upload, diagnostics and XSS endpoints.

_time	method	uri	status	_raw
2026-09-11 10:32:40	POST	/login	200	100.85.187.114 - - [11/Sep/2026:10:32:40 -0400] "POST /login HTTP/1.1" 200 844 "http://100.64.248.91/login" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 10:49:39	POST	/login	200	100.85.187.114 - - [11/Sep/2026:10:49:39 -0400] "POST /login HTTP/1.1" 200 1806 "-" "curl/8.18.0"
2026-09-11 10:59:51	POST	/login	200	100.85.187.114 - - [11/Sep/2026:10:59:51 -0400] "POST /login HTTP/1.1" 200 1820 "-" "curl/8.18.0"
2026-09-11 11:00:34	POST	/login	200	100.85.187.114 - - [11/Sep/2026:11:00:34 -0400] "POST /login HTTP/1.1" 200 859 "http://100.64.248.91/login" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:01:27	POST	/login	302	100.85.187.114 - - [11/Sep/2026:11:01:27 -0400] "POST /login HTTP/1.1" 302 487 "-" "curl/8.18.0"
2026-09-11 11:01:51	POST	/login	302	100.85.187.114 - - [11/Sep/2026:11:01:51 -0400] "POST /login HTTP/1.1" 302 543 "http://100.64.248.91/login" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:02:34	POST	/login	200	100.85.187.114 - - [11/Sep/2026:11:02:34 -0400] "POST /login HTTP/1.1" 200 1806 "-" "curl/8.18.0"
2026-09-11 11:04:19	POST	/documents	200	100.85.187.114 - - [11/Sep/2026:11:04:19 -0400] "POST /documents HTTP/1.1" 200 1449 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:05:05	POST	/documents	200	100.85.187.114 - - [11/Sep/2026:11:05:05 -0400] "POST /documents HTTP/1.1" 200 1462 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:06:25	POST	/documents	200	100.85.187.114 - - [11/Sep/2026:11:06:25 -0400] "POST /documents HTTP/1.1" 200 1462 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:08:40	POST	/documents	200	100.85.187.114 - - [11/Sep/2026:11:08:40 -0400] "POST /documents HTTP/1.1" 200 1462 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:08:48	POST	/documents	200	100.85.187.114 - - [11/Sep/2026:11:08:48 -0400] "POST /documents HTTP/1.1" 200 1462 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:11:46	POST	/diagnostics	200	100.85.187.114 - - [11/Sep/2026:11:11:46 -0400] "POST /diagnostics HTTP/1.1" 200 1490 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:12:20	POST	/diagnostics	200	100.85.187.114 - - [11/Sep/2026:11:12:20 -0400] "POST /diagnostics HTTP/1.1" 200 1467 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"

Figure 4.8 – POST Activity — Login, Upload and Diagnostics

Evidence interpretation. The first POST result set shows repeated login requests followed by POST /documents and the start of POST /diagnostics activity. The sequence supports the reconstructed transition from access attempts to authenticated functions.

2026-09-11 11:12:57	POST	/diagnostics	200	100.85.187.114 - - [11/Sep/2026:11:12:57 -0400] "POST /diagnostics HTTP/1.1" 200 1622 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:13:37	POST	/diagnostics	200	100.85.187.114 - - [11/Sep/2026:11:13:37 -0400] "POST /diagnostics HTTP/1.1" 200 1465 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:14:06	POST	/diagnostics	200	100.85.187.114 - - [11/Sep/2026:11:14:06 -0400] "POST /diagnostics HTTP/1.1" 200 1481 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:15:42	POST	/diagnostics	200	100.85.187.114 - - [11/Sep/2026:11:15:42 -0400] "POST /diagnostics HTTP/1.1" 200 1621 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:18:21	POST	/xss	200	100.85.187.114 - - [11/Sep/2026:11:18:21 -0400] "POST /xss HTTP/1.1" 200 1621 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:19:24	POST	/xss	200	100.85.187.114 - - [11/Sep/2026:11:19:24 -0400] "POST /xss HTTP/1.1" 200 1621 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"

Figure 4.9 – POST Activity — Diagnostics and XSS

Evidence interpretation. The continuation shows later diagnostics submissions and XSS endpoint activity. These are request-level observations; exact form-body payloads are not present in the Apache access log.

Show: 20 Per Page	Format	Preview: On	< Prev 1 2 Next >		
_time	method	uri	status	_raw	
2026-09-11 11:20:08	POST	/xss	200	100.85.187.114 - - [11/Sep/2026:11:20:08 -0400] "POST /xss HTTP/1.1" 200 1621 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"	
2026-09-11 11:25:23	POST	/login	302	100.85.187.114 - - [11/Sep/2026:11:25:23 -0400] "POST /login HTTP/1.1" 302 487 "-" "curl/8.18.0"	

Figure 4.10 – POST Activity — Final Events in the Sequence

Evidence interpretation. The final POST rows complete the request-level chronology, including late XSS/login activity. The screenshot is retained to preserve the full SIEM evidence set rather than relying on a partial table.

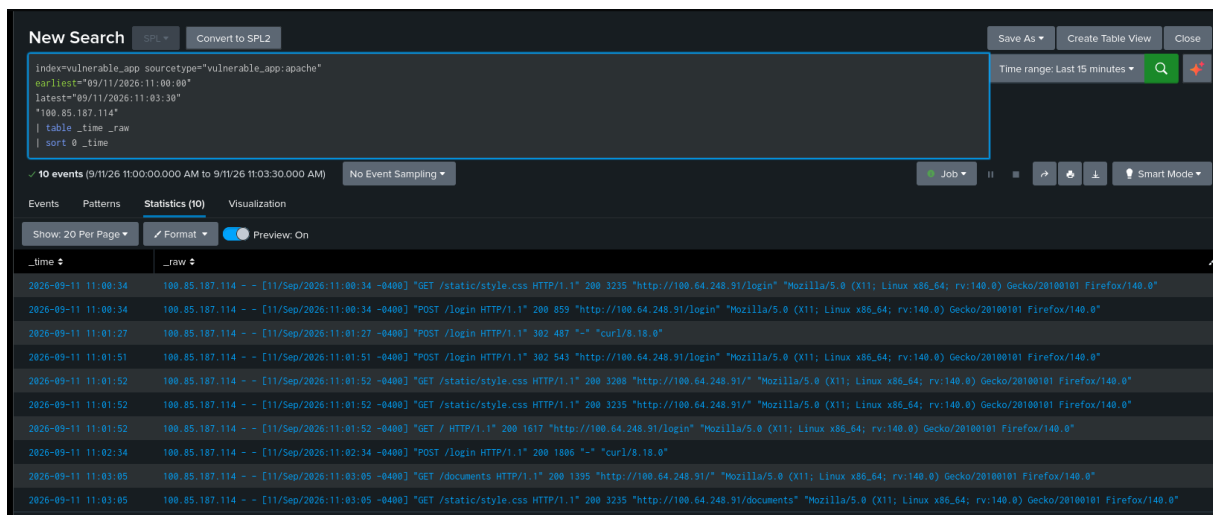


Figure 4.11 – Focused Chronology Around the Authentication Transition

Evidence interpretation. The scoped 11:00–11:03:30 search shows login requests, redirects and the first subsequent application navigation. Redirects are treated as chronology evidence, not by themselves as proof of a successful exploit.

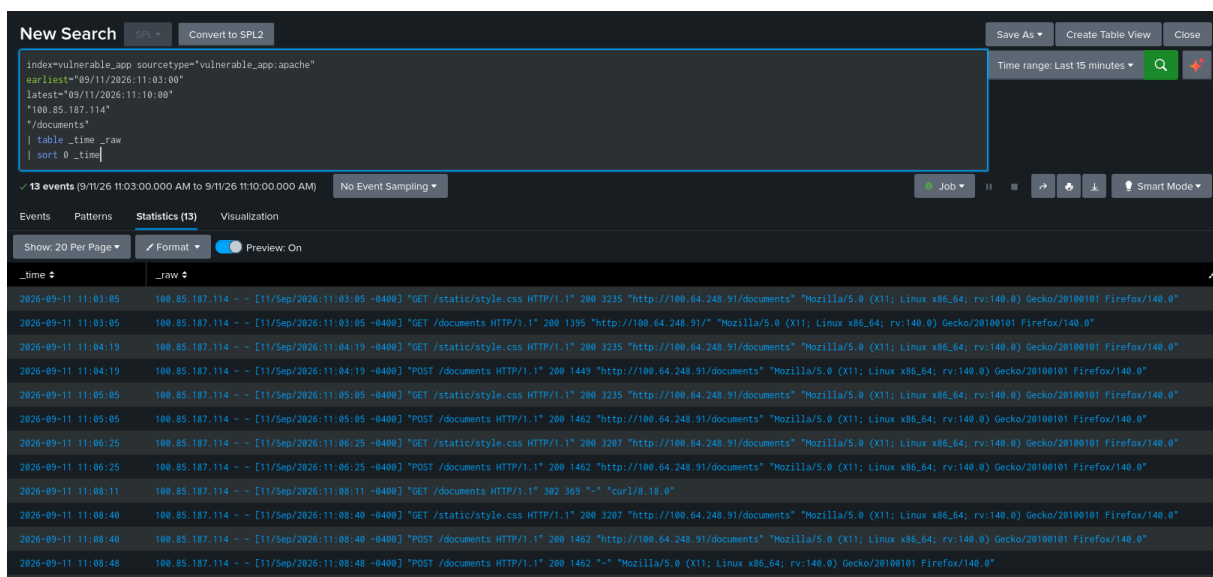


Figure 4.12 – Focused File-Upload Activity Window

Evidence interpretation. The narrower 11:03–11:10 view captures repeated access to /documents and the associated POST requests. These timestamps are later correlated with files identified on VM1.

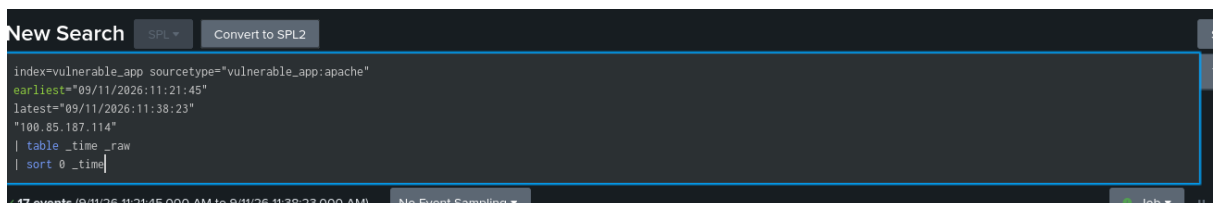


Figure 4.13 – Players Endpoint Investigation Query

Evidence interpretation. The later attack-window search focuses on /players activity where SQL-injection-style search values were sent in the request URI and were therefore observable in Apache telemetry.

_time	_raw
2026-09-11 11:21:45	100.85.187.114 - - [11/Sep/2026:11:21:45 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/teams" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:21:45	100.85.187.114 - - [11/Sep/2026:11:21:45 -0400] "GET /teams HTTP/1.1" 200 1306 "http://100.64.248.91/matches" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:22:33	100.85.187.114 - - [11/Sep/2026:11:22:33 -0400] "GET /players HTTP/1.1" 200 1537 "http://100.64.248.91/teams" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:22:34	100.85.187.114 - - [11/Sep/2026:11:22:34 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/players" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:23:15	100.85.187.114 - - [11/Sep/2026:11:23:15 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/players?search=Ahmad" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:23:15	100.85.187.114 - - [11/Sep/2026:11:23:15 -0400] "GET /players?search=Ahmad HTTP/1.1" 200 1368 "http://100.64.248.91/players" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:24:08	100.85.187.114 - - [11/Sep/2026:11:24:08 -0400] "GET /players HTTP/1.1" 302 369 "-" "curl/8.18.0"
2026-09-11 11:24:28	100.85.187.114 - - [11/Sep/2026:11:24:28 -0400] "GET /players HTTP/1.1" 302 369 "-" "curl/8.18.0"
2026-09-11 11:25:00	100.85.187.114 - - [11/Sep/2026:11:25:00 -0400] "GET /players HTTP/1.1" 302 369 "-" "curl/8.18.0"
2026-09-11 11:25:23	100.85.187.114 - - [11/Sep/2026:11:25:23 -0400] "POST /login HTTP/1.1" 302 487 "-" "curl/8.18.0"
2026-09-11 11:25:48	100.85.187.114 - - [11/Sep/2026:11:25:48 -0400] "GET /players HTTP/1.1" 200 7430 "-" "curl/8.18.0"
2026-09-11 11:26:08	100.85.187.114 - - [11/Sep/2026:11:26:08 -0400] "GET /players?search=27 HTTP/1.1" 200 3793 "-" "curl/8.18.0"
2026-09-11 11:26:35	100.85.187.114 - - [11/Sep/2026:11:26:35 -0400] "GET /players?search=27X200R201X3D1520--X20 HTTP/1.1" 200 7446 "-" "curl/8.18.0"
2026-09-11 11:26:50	100.85.187.114 - - [11/Sep/2026:11:26:50 -0400] "GET /players?search=27X200R201X3D2520--X20 HTTP/1.1" 200 7446 "-" "curl/8.18.0"
2026-09-11 11:27:19	100.85.187.114 - - [11/Sep/2026:11:27:19 -0400] "GET /players?search=Ahmad HTTP/1.1" 200 4304 "-" "curl/8.18.0"
2026-09-11 11:27:23	100.85.187.114 - - [11/Sep/2026:11:27:23 -0400] "GET /players?search=27 HTTP/1.1" 200 3717 "-" "curl/8.18.0"
2026-09-11 11:27:42	100.85.187.114 - - [11/Sep/2026:11:27:42 -0400] "GET /players?search=27X200UNION2SELECTX20NULL--X20 HTTP/1.1" 200 3814 "-" "curl/8.18.0"

Figure 4.14 – Players Search Requests and SQLi-Style Inputs

Evidence interpretation. The raw /players events include encoded search parameters and establish the sequence leading into single-quote, Boolean and UNION SELECT probing.

New Search

```
index=vulnerable_app sourcetype=vulnerable_app:apache
earliest="09/11/2026:10:29:15"
latest="09/11/2026:11:38:23"
| rex "(?<src_ip>S+)"
| rex "\(\?method\>S\)" (?<uri>S+) HTTP/\? (?<status>S+)"
| stats count AS total_requests
  dc(uri) AS unique_uris
  count(eval(status="404")) AS not_found
  by src_ip
| where total_requests > 100 AND unique_uris > 50 AND not_found > 50
| sort - total_requests
```

47780 events (9/11/26 10:29:15.000 AM to 9/11/26 11:38:23.000 AM) No Event Sampling

src_ip	total_requests	unique_uris	not_found
100.85.187.114	47728	43776	47687

Figure 4.15 – Behavioral Web Enumeration Detection

Evidence interpretation. The behavioral rule aggregates requests by source and identifies VM3 as the high-volume source. It supports automated fuzzing detection independently of a specific FFUF user-agent string.

4.2.2 Shell History Analysis – Commands Executed Post-Compromise

The monitored shell history was /home/kali/.zsh_history because Kali used Zsh. Pre-attack benign commands prove that the source was configured and ingesting correctly. No Zsh-history events appeared inside the official attack window, and web-executed commands were not present in the interactive user history. This is a visibility limitation, not evidence that command execution did not occur; the independent Red Team reverse-shell evidence confirms that RCE was achieved.

SHELL-HISTORY VISIBILITY EVIDENCE

New Search

```
index=vulnerable_app
earliest="09/11/2026:10:29:15"
latest="09/11/2026:11:38:23"
| stats count by sourcetype
```

47780 events (9/11/26 10:29:15.000 AM to 9/11/26 11:38:23.000 AM) No Event Sampling

sourcetype	count
vulnerable_app:apache	47780

Figure 4.16 – Attack-Window Log Source Availability

Evidence interpretation. The attack-window search shows Apache telemetry while no Zsh-history activity is returned for the same interval. This documents the shell-history visibility gap rather than treating it as proof that no commands ran.

4.2.3 Correlated Attack Timeline

Table 4.2 – Correlated Original Attack Timeline

Time EDT	Activity	Evidence / interpretation
10:29:15	Investigation window opens	Recorded attacker start time.
10:32:40	Login activity	First POST /login in the scoped endpoint summary.
10:37:02–10:58:22	FFUF enumeration	47,621 requests; overwhelming majority returned 404.
11:01:27 onward	Application access	Protected functions are accessed after login activity; redirects alone are not treated as proof.
11:04:19–11:08:48	File uploads	Five POST /documents events; two filesystem artifacts later correlate to these times.
11:11:46–11:15:42	Diagnostics activity	Six POST /diagnostics requests; POST bodies unavailable.
11:18:21–11:20:08	XSS activity	Three POST /xss requests; POST bodies unavailable.
11:26:08–11:27:42	SQLi probes	/players?search= inputs include quote, Boolean, and UNION SELECT indicators.
11:38:23	Window closes	Recorded attacker stop time.

CORRELATED TIMELINE EVIDENCE

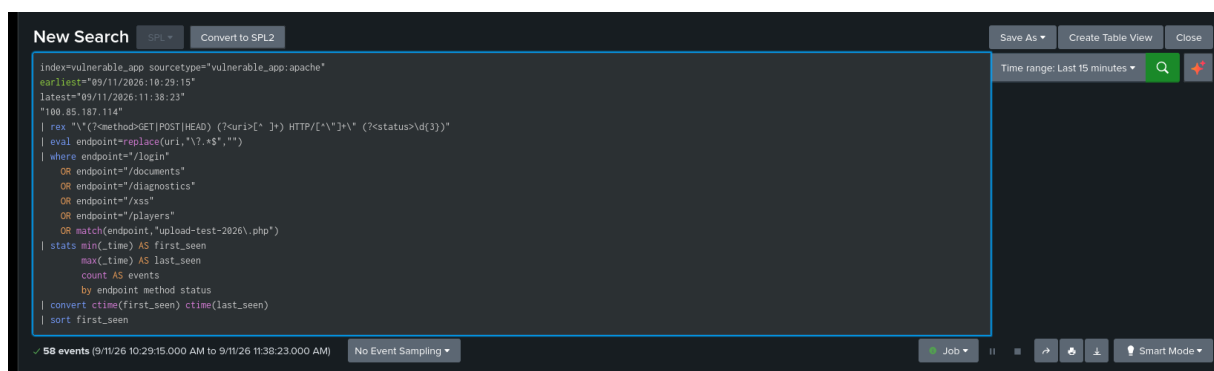


Figure 4.17 – Endpoint Timeline Correlation Query

Evidence interpretation. The timeline query normalizes key application endpoints and calculates first seen, last seen and event counts. It provides a compact reconstruction of the attack sequence.

endpoint	method	status	first_seen	last_seen	events
/login	POST	200	09/11/2026 10:32:40	09/11/2026 11:02:34	5
/login	GET	200	09/11/2026 10:35:40	09/11/2026 10:55:53	7
/diagnostics	GET	302	09/11/2026 10:37:09	09/11/2026 10:56:04	4
/documents	GET	302	09/11/2026 10:37:09	09/11/2026 11:08:11	4
/players	GET	302	09/11/2026 10:37:15	09/11/2026 11:25:00	6
/login	POST	302	09/11/2026 11:01:27	09/11/2026 11:25:23	3
/documents	GET	200	09/11/2026 11:03:05	09/11/2026 11:11:04	2
/documents	POST	200	09/11/2026 11:04:19	09/11/2026 11:08:48	5
/documents/upload-test-2026.php	GET	404	09/11/2026 11:10:01	09/11/2026 11:10:01	1
/players	GET	200	09/11/2026 11:11:01	09/11/2026 11:27:42	10
/diagnostics	GET	200	09/11/2026 11:11:06	09/11/2026 11:11:06	1
/diagnostics	POST	200	09/11/2026 11:11:46	09/11/2026 11:15:42	6
/xss	GET	200	09/11/2026 11:17:41	09/11/2026 11:17:41	1
/xss	POST	200	09/11/2026 11:18:21	09/11/2026 11:20:08	3

Figure 4.18 – Endpoint Timeline Statistics

Evidence interpretation. The statistics output shows the ordered endpoints, request methods, response status, first/last times and counts used in the correlated incident timeline.

4.3 Containment and Remediation

4.3.1 SSH Access to VM1

After the SIEM timeline was established, VM1 was inspected through authorized SSH. An initial time-bounded search under /var/www returned no files, but this directory was not the application location. The application was under /home/kali/vulnerable-app, so later artifact searches were correctly scoped there. Process triage also verified an unusual-looking /proc executable as /usr/bin/qterminal rather than classifying it as malware without evidence.

HOST ACCESS AND NETWORK EVIDENCE

```
(kali㉿kali)-[~]
$ ssh kali@100.64.248.91
The authenticity of host '100.64.248.91 (100.64.248.91)' can't be established.
ED25519 key fingerprint is: SHA256:Q268sLmQpE6p/ChEuy6eoz+aL7IJGL4n1I3dEmgbKFM
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '100.64.248.91' (ED25519) to the list of known hosts.
kali@100.64.248.91's password:
Linux kali 7.1.5+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 7.1.5-1kali1 (2026-07-29) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
(kali㉿kali)-[~]
$ hostname
kali

(kali㉿kali)-[~]
$ whoami
kali

(kali㉿kali)-[~]
$ date
Fri Sep 11 12:30:43 PM EDT 2026
```

Figure 4.19 – Authorized SSH Access to VM1

Evidence interpretation. The investigator connects to VM1 at 100.64.248.91 and verifies the host identity and time. This marks the transition from SIEM triage to host-based incident response.

```
(kali㉿kali)-[~]
$ sudo ss -tunap
Netid      State      Recv-Q     Send-Q     Local Address:Port      Peer Address:Port
Process
udp        UNCONN     0           0           0.0.0.0:41641            0.0.0.0:*
users:((("tailscaled",pid=652,fd=20))
udp        ESTAB      0           0           10.0.2.15%eth0:68        10.0.2.2:67
users:((("NetworkManager",pid=483,fd=26))
udp        UNCONN     0           0           [::]:41641              [::]:*
users:((("tailscaled",pid=652,fd=19))
tcp        LISTEN     0           2048        127.0.0.1:5000           0.0.0.0:*
users:((("gunicorn",pid=38347,fd=3),("gunicorn",pid=38346,fd=3),("gunicorn",pid=38335,fd=3))
tcp        LISTEN     0           4096        100.64.248.91:46875      0.0.0.0:*
users:((("tailscaled",pid=652,fd=23))
tcp        LISTEN     0           128         0.0.0.0:22              0.0.0.0:*
users:((("sshd",pid=114508,fd=6))
tcp        ESTAB      0           0           10.0.2.15:47666         199.165.136.100:443
users:((("tailscaled",pid=652,fd=9))
tcp        ESTAB      0           880         100.64.248.91:22        100.81.218.80:53678
users:((("sshd-session",pid=114632,fd=7),("sshd-session",pid=114592,fd=7))
tcp        TIME-WAIT  0           0           100.64.248.91:45484     100.81.218.80:9997
tcp        ESTAB      0           0           10.0.2.15:58772         192.200.0.101:443
users:((("tailscaled",pid=652,fd=18))
tcp        ESTAB      0           0           100.64.248.91:53208     100.81.218.80:9997
users:((("splunkd",pid=719,fd=68))
tcp        ESTAB      0           0           10.0.2.15:49716         185.34.3.75:443
users:((("tailscaled",pid=652,fd=21))
tcp        TIME-WAIT  0           0           100.64.248.91:55510     100.81.218.80:9997
tcp        LISTEN     0           4096        [fd7a:115c:a1e0::2501:f8c5]:61103 [::]:*
users:((("tailscaled",pid=652,fd=24))
tcp        LISTEN     0           128         [::]:22                  [::]:*
users:((("sshd",pid=114508,fd=7))
tcp        LISTEN     0           511         *:80                     *:80
users:((("apache2",pid=68519,fd=4),("apache2",pid=68516,fd=4),("apache2",pid=68515,fd=4),("apache2",pid=68509,fd=4),("apache2",pid=68494,fd=4),
("apache2",pid=68488,fd=4),("apache2",pid=68477,fd=4),("apache2",pid=68464,fd=4),("apache2",pid=68455,fd=4),("apache2",pid=54901,fd=4),("apac
he2",pid=30824,fd=4))
```

Figure 4.20 – Post-Incident Network Connection Inspection

Evidence interpretation. The socket review shows expected SSH and Splunk-forwarding connections at inspection time and no obvious active reverse-shell connection. This is a point-in-time observation only.

4.3.2 Webshell / Artifact Identification and Deletion

A time-bounded search of /home/kali/vulnerable-app identified two files in uploads: upload-test-2026.txt and upload-test-2026.php. They were investigated as attack-window artifacts. The PHP file contained only a benign echo test and direct execution was not demonstrated, so the report does not call it a confirmed webshell.

Table 4.3 – Identified Upload Artifacts

Artifact	Type / size	SHA-256	Assessment
upload-test-2026.txt	ASCII text · 33 bytes	2cfb0dbd426e64dfa6b4259172871cbff29b1a9994c63564b125819704c7dad0	Upload test marker.
upload-test-2026.php	PHP script / ASCII text · 34 bytes	9c36e19f965c484d70e0d7deb1debfa24c87d07684f40d86a24b43ab03257053	PHP echo test artifact; not a confirmed webshell.

The text file timestamp aligned with POST /documents at 11:04:19. The PHP file birth time aligned with a POST at 11:05:05, and its later modification aligned with the 11:08:48 POST. This timestamp correlation links the filesystem artifacts to the web upload activity without relying on filenames in the Apache body, which was unavailable.

ARTIFACT INVESTIGATION AND CONTAINMENT EVIDENCE



Figure 4.21 – Suspicious PHP File Access Investigation

Evidence interpretation. Splunk is used to inspect access to the uploaded PHP test path. The observed HTTP result is retained as request evidence; it does not establish execution of the PHP content.

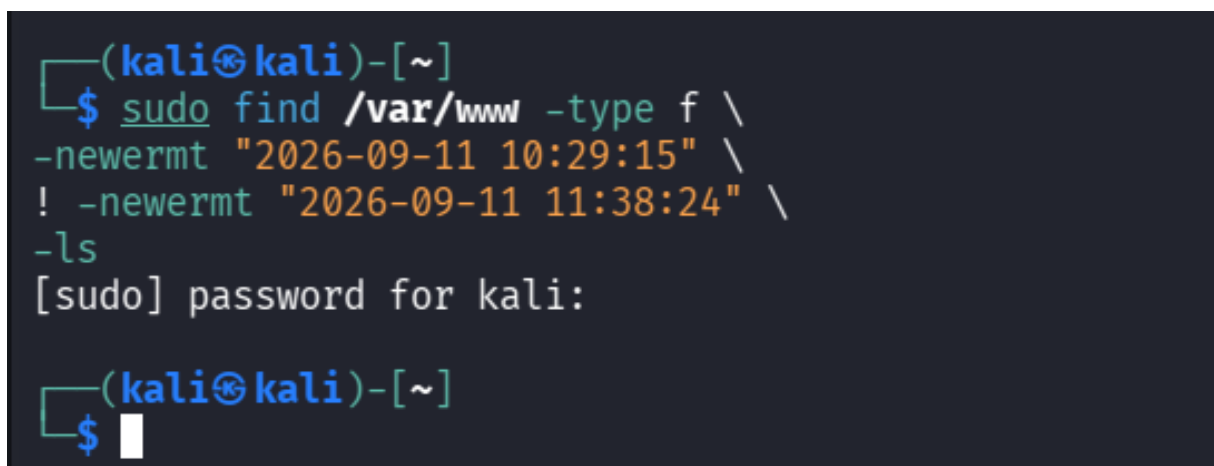


Figure 4.22 – Initial Web-Root Attack-Window File Search

Evidence interpretation. A time-bounded search of /var/www returns no matching files. The result prompted the investigator to inspect the actual application path under /home/kali/vulnerable-app.

```

splunkf+ 719 0.2 3.1 379200 64400 ? Ssl 08:46 0:40 splunkd --under-systemd --systemd-delegate=no -p 8089 _interna
r_systemd
kali 33864 0.0 2.9 2435404 59704 ? Sl 09:47 0:07 /usr/lib/firefox-esr/firefox-esr -contentproc -isForBrowser -p
40590 -prefMapHandle 1:272002 -jsInitHandle 2:242832 -parentBuildID 20260811190631 -sandboxReporter 3 -chrootClient 4 -ipcHandle
nnelId {9e4b0b31-6735-4fc5-975e-271a865deebc} -parentPid 12289 -crashReporter 6 -crashHelper 7 -greomni /usr/lib/firefox-esr/omni
/usr/lib/firefox-esr/browser/omni.ja -appDir /usr/lib/firefox-esr/browser 17 tab
kali 33854 0.0 2.9 2435404 59652 ? Sl 09:47 0:07 /usr/lib/firefox-esr/firefox-esr -contentproc -isForBrowser -p
40590 -prefMapHandle 1:272002 -jsInitHandle 2:242832 -parentBuildID 20260811190631 -sandboxReporter 3 -chrootClient 4 -ipcHandle
nnelId {ec59112c-e2e9-4d5e-8c0b-dec238360368} -parentPid 12289 -crashReporter 6 -crashHelper 7 -greomni /usr/lib/firefox-esr/omni
/usr/lib/firefox-esr/browser/omni.ja -appDir /usr/lib/firefox-esr/browser 16 tab
kali 33917 0.0 2.9 2435404 59556 ? Sl 09:47 0:07 /usr/lib/firefox-esr/firefox-esr -contentproc -isForBrowser -p
40590 -prefMapHandle 1:272002 -jsInitHandle 2:242832 -parentBuildID 20260811190631 -sandboxReporter 3 -chrootClient 4 -ipcHandle
nnelId {d1ccc192-f7ee-4f96-b274-2ec4fc33bbb6} -parentPid 12289 -crashReporter 6 -crashHelper 7 -greomni /usr/lib/firefox-esr/omni
/usr/lib/firefox-esr/browser/omni.ja -appDir /usr/lib/firefox-esr/browser 18 tab
kali 48830 0.0 2.9 665148 58980 ? Sl 10:16 0:05 /proc/2175/exe
kali 1440 0.0 2.5 991112 51736 ? Sl 08:46 0:05 xfce4-panel
kali 43878 0.0 2.2 663896 45824 ? Sl 10:06 0:02 /usr/bin/qterminal
kali 1455 0.0 2.2 1044332 44812 ? Sl 08:46 0:07 xfdesktop
kali 2175 0.5 2.1 676304 42988 ? Sl 08:46 1:14 /usr/bin/qterminal

(kali@kali)-[~]
$ ps -fp 48830
readlink -f /proc/48830/exe
cat /proc/48830/cmdline | tr '\0' ' '; echo
ps -o pid,ppid,user,lstart,cmd -p 48830
UID      PID      PPID    C  STIME TTY          TIME CMD
kali     48830      1    0 10:16 ?        00:00:05 /proc/2175/exe
/usr/bin/qterminal
/proc/2175/exe
      PID      PPID USER          STARTED CMD

```

Figure 4.23 – Process Triage and Executable Verification

Evidence interpretation. An unusual-looking /proc executable is resolved to /usr/bin/qterminal. The screenshot demonstrates evidence-based triage and avoids incorrectly labeling a legitimate process as malicious.

```

(kali@kali)-[~]
$ sudo find /home/kali/vulnerable-app -type f \
-newermt "2026-09-11 10:29:15" \
! -newermt "2026-09-11 11:38:24" \
-printf '%TY-%Tm-%Td %TH:%TM:%TS %u %g %m %p\n' \
| sort
2026-09-11 11:04:19.3967380520 kali kali 644 /home/kali/vulnerable-app/uploads/upload-test-2026.txt
2026-09-11 11:08:48.2456754820 kali kali 644 /home/kali/vulnerable-app/uploads/upload-test-2026.php

```

Figure 4.24 – Attack-Window Artifact Search in the Application Directory

Evidence interpretation. The correctly scoped search under /home/kali/vulnerable-app identifies upload-test-2026.txt and upload-test-2026.php within the attack window.

```

(kali@kali)-[~]
$ sudo stat /home/kali/vulnerable-app/uploads/upload-test-2026.txt
sudo stat /home/kali/vulnerable-app/uploads/upload-test-2026.php
File: /home/kali/vulnerable-app/uploads/upload-test-2026.txt
Size: 33      Blocks: 8      IO Block: 4096   regular file
Device: 8,1   Inode: 678895   Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-09-11 11:04:19.392803326 -0400
Modify: 2026-09-11 11:04:19.396738052 -0400
Change: 2026-09-11 11:04:19.396738052 -0400
Birth: 2026-09-11 11:04:19.392803326 -0400
File: /home/kali/vulnerable-app/uploads/upload-test-2026.php
Size: 34      Blocks: 8      IO Block: 4096   regular file
Device: 8,1   Inode: 678919   Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-09-11 11:05:05.724803149 -0400
Modify: 2026-09-11 11:08:48.245675482 -0400
Change: 2026-09-11 11:08:48.245675482 -0400
Birth: 2026-09-11 11:05:05.724803149 -0400

(kali@kali)-[~]
$ sudo file /home/kali/vulnerable-app/uploads/upload-test-2026.txt
sudo file /home/kali/vulnerable-app/uploads/upload-test-2026.php
/home/kali/vulnerable-app/uploads/upload-test-2026.txt: ASCII text
/home/kali/vulnerable-app/uploads/upload-test-2026.php: PHP script, ASCII text

(kali@kali)-[~]
$ sudo sed -n '1,120p' /home/kali/vulnerable-app/uploads/upload-test-2026.txt
sudo sed -n '1,120p' /home/kali/vulnerable-app/uploads/upload-test-2026.php
File upload security test - Hadi
<?php echo "FILE_UPLOAD_TEST"; ?>

```

Figure 4.25 – Artifact Metadata, Type and Content Review

Evidence interpretation. stat, file and content inspection document timestamps, permissions, file types and the harmless test content before containment. The PHP file is therefore treated as an uploaded test artifact, not a confirmed webshell.

```

(kali@kali)-[~]
$ sudo sha256sum \
/home/kali/vulnerable-app/uploads/upload-test-2026.txt \
/home/kali/vulnerable-app/uploads/upload-test-2026.php
2cfb0dbd426e64dfa6b4259172871cbff29b1a9994c63564b125819704c7dad0 /home/kali/vulnerable-app/uploads/upload-test-2026.txt
9c36e19f965c484d70e0d7deb1debfa24c87d07684f40d86a24b43ab03257053 /home/kali/vulnerable-app/uploads/upload-test-2026.php

```

Figure 4.26 – SHA-256 Hashes of Identified Artifacts

Evidence interpretation. SHA-256 values are recorded for both upload artifacts before deletion, preserving identifiers for the incident record.

```

New Search
index=vulnerable_app sourcetype="vulnerable_app:apache"
earliest="09/11/2026:11:03:30"
latest="09/11/2026:11:09:30"
"100.85.187.114"
| rex "\(<method>GET|POST|HEAD\) (?<uri>[^\s]+) HTTP/[^\s]+\)" (?<status>\d{3})"
| table _time method uri status _raw
| sort 0 _time

```

Figure 4.27 – HTTP-to-Filesystem Correlation Query

Evidence interpretation. A focused Splunk search retrieves /documents activity during the upload period so HTTP timestamps can be compared with the host filesystem timestamps.

_time	method	uri	status	_raw
2026-09-11 11:04:19	GET	/static/style.css	200	100.85.187.114 - - [11/Sep/2026:11:04:19 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:04:19	POST	/documents	200	100.85.187.114 - - [11/Sep/2026:11:04:19 -0400] "POST /documents HTTP/1.1" 200 1449 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:05:05	GET	/static/style.css	200	100.85.187.114 - - [11/Sep/2026:11:05:05 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:05:05	POST	/documents	200	100.85.187.114 - - [11/Sep/2026:11:05:05 -0400] "POST /documents HTTP/1.1" 200 1462 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:06:25	GET	/static/style.css	200	100.85.187.114 - - [11/Sep/2026:11:06:25 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:06:25	POST	/documents	200	100.85.187.114 - - [11/Sep/2026:11:06:25 -0400] "POST /documents HTTP/1.1" 200 1462 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:08:11	GET	/documents	302	100.85.187.114 - - [11/Sep/2026:11:08:11 -0400] "GET /documents HTTP/1.1" 302 369 "-" "curl/8.18.0"
2026-09-11 11:08:40	GET	/static/style.css	200	100.85.187.114 - - [11/Sep/2026:11:08:40 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:08:40	POST	/documents	200	100.85.187.114 - - [11/Sep/2026:11:08:40 -0400] "POST /documents HTTP/1.1" 200 1462 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:08:48	POST	/documents	200	100.85.187.114 - - [11/Sep/2026:11:08:48 -0400] "POST /documents HTTP/1.1" 200 1462 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:08:48	GET	/static/style.css	200	100.85.187.114 - - [11/Sep/2026:11:08:48 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"

Figure 4.28 – HTTP Upload Events Used for Artifact Correlation

Evidence interpretation. The POST /documents events include the 11:04:19, 11:05:05 and later upload times that align with the creation/modification timestamps recorded on VM1.

```
(kali㉿kali)-[~]
$ sudo rm -- /home/kali/vulnerable-app/uploads/upload-test-2026.txt
sudo rm -- /home/kali/vulnerable-app/uploads/upload-test-2026.php

(kali㉿kali)-[~]
$ sudo ls -la /home/kali/vulnerable-app/uploads/
total 20
drwxrwxr-x 2 kali kali 4096 Sep 11 12:44 .
drwxrwxr-x 9 kali kali 4096 Sep 11 10:06 ..
-rw-r--r-- 1 kali kali 16 Sep 5 18:42 lab-test.txt
-rw-r--r-- 1 kali kali 33 Sep 7 13:34 poc.php
-rw-r--r-- 1 kali kali 99 Sep 7 13:25 shell.php
-rw-r--r-- 1 kali kali 0 Sep 7 13:34 test.txt
```

Figure 4.29 – Containment — Removal of Upload Artifacts

Evidence interpretation. The two identified attack-window artifacts are removed from the uploads directory, after which the directory contents are listed to verify the targeted files are gone.

```
(kali㉿kali)-[~]
$ sudo find /home/kali/vulnerable-app -type f \
-newermt "2026-09-11 10:29:15" \
! -newermt "2026-09-11 11:38:24" \
-printf '%TY-%Tm-%Td %TH:%TM:%TS %p\n' \
| sort

(kali㉿kali)-[~]
$
```

Figure 4.30 – Post-Containment Attack-Window File Search

Evidence interpretation. The same time-bounded application-directory search is repeated after containment and returns no remaining files from the defined incident window.

4.3.3 Persistence Check and Clearance Confirmation

The inspected persistence checks covered root and user crontabs, cron directories, enabled systemd services, and user autostart. No obvious persistence was identified during these checks. This is deliberately scoped wording: the reviewed locations did not contain an obvious malicious mechanism, but the evidence does not prove that every possible persistence technique was absent.

PERSISTENCE REVIEW EVIDENCE

```

(kali@kali)-[~]
$ sudo crontab -l
crontab -l
sudo ls -la /etc/cron.d /etc/cron.daily /etc/cron.hourly
no crontab for root
no crontab for kali
/etc/cron.d:
total 36
drwxr-xr-x  2 root root  4096 Sep  5 16:38 .
drwxr-xr-x 187 root root 12288 Sep 11 08:46 ..
-rw-r--r--  1 root root   188 Mar  6 2026 e2scrub_all
-rw-r--r--  1 root root   607 Jun  3 2025 john
-rw-r--r--  1 root root   712 Dec 20 2025 php
-rw-r--r--  1 root root   102 Apr 27 03:56 .placeholder
-rw-r--r--  1 root root   400 Jan 15 2024 sysstat

/etc/cron.daily:
total 48
drwxr-xr-x  2 root root  4096 Sep  5 16:18 .
drwxr-xr-x 187 root root 12288 Sep 11 08:46 ..
-rwxr-xr-x  1 root root   539 Dec  5 2025 apache2
-rwxr-xr-x  1 root root  1478 Apr 14 03:24 apt-compat
-rwxr-xr-x  1 root root   123 Mar 27 05:30 dpkg
-rwxr-xr-x  1 root root   377 Jul 14 2024 logrotate
-rwxr-xr-x  1 root root  1395 May  2 2025 man-db
-rw-r--r--  1 root root   102 Apr 27 03:56 .placeholder
-rwxr-xr-x  1 root root   652 Dec  7 2020 plocate
-rwxr-xr-x  1 root root   526 Jan 15 2024 sysstat

/etc/cron.hourly:
total 20
drwxr-xr-x  2 root root  4096 Jun 16 10:23 .
drwxr-xr-x 187 root root 12288 Sep 11 08:46 ..
-rw-r--r--  1 root root   102 Apr 27 03:56 .placeholder

```

Figure 4.31 – Cron Persistence Review

Evidence interpretation. Root/user crontab state and cron directories are inspected for unauthorized scheduled execution. No obvious malicious persistence is identified in the displayed checks.

```
(kali@kali)-[~]
$ sudo systemctl list-unit-files --type=service --state=enabled
```

UNIT	FILE	STATE	PRESET
accounts-daemon.service		enabled	enabled
apache2.service		enabled	disabled
console-setup.service		enabled	enabled
cron.service		enabled	enabled
getty@.service		enabled	enabled
grub-install-devices.service		enabled	disabled
haveged.service		enabled	enabled
keyboard-setup.service		enabled	enabled
lightdm.service		enabled	disabled
ModemManager.service		enabled	enabled
networking.service		enabled	enabled
NetworkManager-dispatcher.service		enabled	disabled
NetworkManager-wait-online.service		enabled	disabled
NetworkManager.service		enabled	enabled
regenerate-ssh-host-keys.service		enabled	enabled
smartmontools.service		enabled	enabled
SplunkForwarder.service		enabled	disabled
systemd-pstore.service		enabled	enabled
systemd-timesyncd.service		enabled	enabled
tailscaled.service		enabled	disabled
virtualbox-guest-utils.service		enabled	enabled
vulnerable-app.service		enabled	disabled

```
22 unit files listed.
```

Figure 4.32 – Enabled systemd Service Review

Evidence interpretation. Enabled services are reviewed for suspicious persistence. No obvious malicious service is identified during this scoped check.

4.4 Detection Engineering

The four core Splunk searches use index `vulnerable_app` and sourcetype `vulnerable_app:apache`. Their purpose is to identify attack attempts, not to prove exploitation. The upload, XSS, and Diagnostics detections are primarily endpoint-based because Apache did not capture the form POST body. SQL injection has stronger payload-level visibility where the probe value appears in the URI.

4.4.1 Suspicious File Upload Detection

```
index=vulnerable_app sourcetype="vulnerable_app:apache"
| rex "^(<src_ip>\S+)"
| rex "\"(<method>\S+) (<uri>\S+) HTTP/[\^\" ]+\" (<status>\d{3})\""
| where method="POST" AND like(uri,"/documents%")
| table _time src_ip method uri status
| sort 0 _time
```

A match proves that an upload request reached `/documents`. Acceptance or storage must be confirmed with application or filesystem evidence.

DETECTION EVIDENCE — FILE UPLOAD

New Search SPL Convert to SPL2 Save As Create Table View Close

Time range: Last 15 minutes Q +

```
index=vulnerable_app sourcetype='vulnerable_app:apache'
earliest=09/11/2026:10:29:15
latest=09/11/2026:11:38:23
| rex \"(?<method>S+) (?<uri>S+) HTTP/[^\"]+\" (?<status>d{3}) (?<bytes>S+)\"
| where uri=\"/documents\" AND method=\"POST\"
| table _time method uri status bytes _raw
| sort 0 _time
```

5 Events (9/11/26 10:29:15.000 AM to 9/11/26 11:38:23.000 AM) No Event Sampling Job || Smart Mode

Events Patterns **Statistics (5)** Visualization

Show: 20 Per Page Format Preview: On

_time	method	uri	status	bytes	_raw
2026-09-11 11:04:19	POST	/documents	200	1449	100.85.187.114 - - [11/Sep/2026:11:04:19 -0400] "POST /documents HTTP/1.1" 200 1449 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:05:05	POST	/documents	200	1462	100.85.187.114 - - [11/Sep/2026:11:05:05 -0400] "POST /documents HTTP/1.1" 200 1462 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:06:25	POST	/documents	200	1462	100.85.187.114 - - [11/Sep/2026:11:06:25 -0400] "POST /documents HTTP/1.1" 200 1462 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:08:48	POST	/documents	200	1462	100.85.187.114 - - [11/Sep/2026:11:08:48 -0400] "POST /documents HTTP/1.1" 200 1462 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:08:48	POST	/documents	200	1462	100.85.187.114 - - [11/Sep/2026:11:08:48 -0400] "POST /documents HTTP/1.1" 200 1462 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"

Figure 4.33 – Original-Window File Upload Detection

Evidence interpretation. The upload detection returns the POST /documents activity from VM3. This confirms the rule fires against the original attack data; host evidence separately proves that two upload artifacts were created.

4.4.2 XSS Vulnerability Detection

```
index=vulnerable_app sourcetype="vulnerable_app:apache"
| rex "^(<src_ip>S+)"
| rex "\"(<method>S+) (<uri>S+) HTTP/[^\"]+\" (<status>d{3})"
| eval decoded_uri=urdecode(uri)
| where (method="POST" AND like(uri, "/xss%"))
| OR match(decoded_uri, "(?i)(<script|javascript:|onerror\s*=|onload\s*=|<svg|<img>)")
| table _time src_ip method uri status decoded_uri
| sort 0 _time
```

The endpoint condition is the supported signal for the observed test because the actual announcement text was in the POST body and was not logged by Apache.

DETECTION EVIDENCE — XSS ACTIVITY

Show: 20 Per Page Format Preview: On < Prev 1 2 Next >

_time	_raw
2026-09-11 11:13:37	100.85.187.114 - - [11/Sep/2026:11:13:37 -0400] "POST /diagnostics HTTP/1.1" 200 1465 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:14:06	100.85.187.114 - - [11/Sep/2026:11:14:06 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:14:06	100.85.187.114 - - [11/Sep/2026:11:14:06 -0400] "POST /diagnostics HTTP/1.1" 200 1481 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:15:42	100.85.187.114 - - [11/Sep/2026:11:15:42 -0400] "POST /diagnostics HTTP/1.1" 200 1621 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:15:52	100.85.187.114 - - [11/Sep/2026:11:15:52 -0400] "GET /static/style.css HTTP/1.1" 200 3208 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"

Figure 4.34 – Transition from Diagnostics to XSS Testing

Evidence interpretation. This result set captures the end of diagnostics activity and the start of requests to /xss. It preserves the ordering between command-injection testing and stored-XSS testing.

New Search SPL Convert to SPL2 Save As Create Table View Close

index=vulnerable_app sourcetype=vulnerable_app:apache
 earliest="09/11/2026:11:15:40"
 latest="09/11/2026:11:22:00"
 "100.85.187.114"
 | table _time _raw
 | sort 0 _time

14 events (9/11/26 11:15:40.000 AM to 9/11/26 11:22:00.000 AM) No Event Sampling Job ▾ Time range: Last 15 minutes Smart Mode ▾

Events Patterns **Statistics (14)** Visualization

Show: 20 Per Page ▾ Format ▾ Preview: On

_time	_raw
2026-09-11 11:15:42	100.85.187.114 -- [11/Sep/2026:11:15:42 -0400] "POST /diagnostics HTTP/1.1" 200 1621 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:15:52	100.85.187.114 -- [11/Sep/2026:11:15:52 -0400] "GET /static/style.css HTTP/1.1" 200 3208 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:17:41	100.85.187.114 -- [11/Sep/2026:11:17:41 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:17:41	100.85.187.114 -- [11/Sep/2026:11:17:41 -0400] "GET /xss HTTP/1.1" 200 1545 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:18:21	100.85.187.114 -- [11/Sep/2026:11:18:21 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:18:21	100.85.187.114 -- [11/Sep/2026:11:18:21 -0400] "POST /xss HTTP/1.1" 200 1621 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:19:24	100.85.187.114 -- [11/Sep/2026:11:19:24 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:19:24	100.85.187.114 -- [11/Sep/2026:11:19:24 -0400] "POST /xss HTTP/1.1" 200 1621 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:20:08	100.85.187.114 -- [11/Sep/2026:11:20:08 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:20:08	100.85.187.114 -- [11/Sep/2026:11:20:08 -0400] "POST /xss HTTP/1.1" 200 1621 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:21:28	100.85.187.114 -- [11/Sep/2026:11:21:28 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/matches" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:21:28	100.85.187.114 -- [11/Sep/2026:11:21:28 -0400] "GET /matches HTTP/1.1" 200 1318 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:21:45	100.85.187.114 -- [11/Sep/2026:11:21:45 -0400] "GET /teams HTTP/1.1" 200 1306 "http://100.64.248.91/matches" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:21:45	100.85.187.114 -- [11/Sep/2026:11:21:45 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/teams" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"

Figure 4.35 – XSS Activity in the Attack Window

Evidence interpretation. The scoped result includes POST /xss requests from VM3. Because Apache did not record POST bodies, the Splunk event confirms the XSS endpoint attempt while browser evidence confirms script execution.

4.4.3 OS Command Injection Detection

```
index=vulnerable_app sourcetype="vulnerable_app:apache"
| rex "(?<src_ip>\S+)"
| rex "\"(?<method>\S+) (?<uri>\S+) HTTP\[^\"]+\\" (?<status>\d{3})"
| eval decoded_uri=uri::decode(uri)
| where (method="POST" AND like(uri,"/diagnostics%"))
OR
match(decoded_uri,"(?i)(;&|\\|'|`|\\$(|\\bwhoami\\b|\\bid\\b|\\buname\\b|\\bcats\\b|\\bcurl\\b|\\bwget\\b|\\bnnc\\b)"))
| table _time src_ip method uri status decoded_uri
| sort 0 _time
```

The result identifies relevant Diagnostics traffic and common visible command indicators. It does not prove that a shell executed; command execution requires supporting application, process, or host evidence.

DETECTION EVIDENCE — DIAGNOSTICS ACTIVITY

New Search SPL Convert to SPL2 Save As Create Table View Close

index=vulnerable_app sourcetype=vulnerable_app:apache
 earliest="09/11/2026:11:10:00"
 latest="09/11/2026:11:17:00"
 "100.85.187.114"
 | table _time _raw
 | sort 0 _time

14 events (9/11/26 11:10:00.000 AM to 9/11/26 11:17:00.000 AM) No Event Sampling Job ▾ Time range: Last 15 minutes Smart Mode ▾

Events Patterns **Statistics (14)** Visualization

Show: 20 Per Page ▾ Format ▾ Preview: On

_time	_raw
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "POST /diagnostics HTTP/1.1" 200 1621 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "GET /static/style.css HTTP/1.1" 200 3208 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "GET /xss HTTP/1.1" 200 1545 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "POST /xss HTTP/1.1" 200 1621 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "POST /xss HTTP/1.1" 200 1621 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "POST /xss HTTP/1.1" 200 1621 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/matches" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "GET /matches HTTP/1.1" 200 1318 "http://100.64.248.91/xss" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "GET /teams HTTP/1.1" 200 1306 "http://100.64.248.91/matches" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:00	100.85.187.114 -- [11/Sep/2026:11:10:00 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/teams" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"

Figure 4.36 – Diagnostics Activity Query Window

Evidence interpretation. The SIEM search scopes VM3 traffic around the period in which the Red Team exercised the System Status function. The request body is not available, so the event proves endpoint activity rather than the exact command string.

_time	_raw
2026-09-11 11:10:01	100.85.187.114 - - [11/Sep/2026:11:10:01 -0400] "GET /documents/upload-test-2026.php HTTP/1.1" 200 404 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:57	100.85.187.114 - - [11/Sep/2026:11:10:57 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:57	100.85.187.114 - - [11/Sep/2026:11:10:57 -0400] "GET / HTTP/1.1" 200 1618 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:59	100.85.187.114 - - [11/Sep/2026:11:10:59 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/teams" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:10:59	100.85.187.114 - - [11/Sep/2026:11:10:59 -0400] "GET /teams HTTP/1.1" 200 1305 "http://100.64.248.91/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:11:01	100.85.187.114 - - [11/Sep/2026:11:11:01 -0400] "GET /players HTTP/1.1" 200 1536 "http://100.64.248.91/teams" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:11:02	100.85.187.114 - - [11/Sep/2026:11:11:02 -0400] "GET /matches HTTP/1.1" 200 1289 "http://100.64.248.91/players" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:11:02	100.85.187.114 - - [11/Sep/2026:11:11:02 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/players" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:11:03	100.85.187.114 - - [11/Sep/2026:11:11:03 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/matches" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:11:04	100.85.187.114 - - [11/Sep/2026:11:11:04 -0400] "GET /documents HTTP/1.1" 200 1420 "http://100.64.248.91/matches" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:11:05	100.85.187.114 - - [11/Sep/2026:11:11:05 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:11:06	100.85.187.114 - - [11/Sep/2026:11:11:06 -0400] "GET /diagnostics HTTP/1.1" 200 1376 "http://100.64.248.91/documents" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:11:06	100.85.187.114 - - [11/Sep/2026:11:11:06 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:11:46	100.85.187.114 - - [11/Sep/2026:11:11:46 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:11:46	100.85.187.114 - - [11/Sep/2026:11:11:46 -0400] "POST /diagnostics HTTP/1.1" 200 1490 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:12:20	100.85.187.114 - - [11/Sep/2026:11:12:20 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:12:20	100.85.187.114 - - [11/Sep/2026:11:12:20 -0400] "POST /diagnostics HTTP/1.1" 200 1467 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:12:57	100.85.187.114 - - [11/Sep/2026:11:12:57 -0400] "GET /static/style.css HTTP/1.1" 200 3207 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:12:57	100.85.187.114 - - [11/Sep/2026:11:12:57 -0400] "POST /diagnostics HTTP/1.1" 200 1622 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
2026-09-11 11:13:37	100.85.187.114 - - [11/Sep/2026:11:13:37 -0400] "GET /static/style.css HTTP/1.1" 200 3235 "http://100.64.248.91/diagnostics" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"

Figure 4.37 – Diagnostics Activity Results

Evidence interpretation. The result rows show repeated POST /diagnostics requests from VM3 during the command-injection test period. Host and Red Team evidence are required to establish actual command execution.

4.4.4 SQL Injection Detection

```
index=vulnerable_app sourcetype="vulnerable_app:apache"
| rex "^(<src_ip>\S+)"
| rex "\"(<method>\S+) (<uri>\S+) HTTP/[^\"]+\" (<status>\d{3})"
| eval decoded_uri=urldecode(uri)
| where
match(decoded_uri,"(?i)(/players)?search=.*('|%27|union\s+select|\sor\s|\sand\s)|union\s+select|information
_schema|sleep\s*\\(|benchmark\s*\\(|)")
| table _time src_ip method uri decoded_uri status
| sort 0 _time
```

The single-quote indicator is intentionally low-specificity and must be interpreted in context. During the controlled test it matched the recorded SQLi-style URI probe; the match itself does not establish database compromise.

DETECTION EVIDENCE — SQL INJECTION

New Search		Convert to SPL2	Save
<pre>index=vulnerable_app sourcetype="vulnerable_app:apache" earliest="09/11/2026:11:27:00" latest="09/11/2026:11:38:23" "100.85.187.114" table _time _raw sort 0 _time</pre>			
✓ 3 events (9/11/26 11:27:00.000 AM to 9/11/26 11:38:23.000 AM) No Event Sampling Job			
Events	Patterns	Statistics (3)	Visualization
Show: 20 Per Page Format Preview: On			
_time	_raw		
2026-09-11 11:27:19	100.85.187.114 - - [11/Sep/2026:11:27:19 -0400] "GET /players?search=Ahmad HTTP/1.1" 200 4304 "-" "curl/8.18.0"		
2026-09-11 11:27:23	100.85.187.114 - - [11/Sep/2026:11:27:23 -0400] "GET /players?search=zzzzzzzz HTTP/1.1" 200 3717 "-" "curl/8.18.0"		
2026-09-11 11:27:42	100.85.187.114 - - [11/Sep/2026:11:27:42 -0400] "GET /players?search=%27%20UNION%20SELECT%20NULL--%20 HTTP/1.1" 200 3814 "-" "curl/8.18.0"		

Figure 4.38 – SQL Injection Probe Events — Focused Result

Evidence interpretation. A focused SQLi interval returns the later /players?search= probes from VM3. These events are useful because the relevant search value appears directly in the request URI.

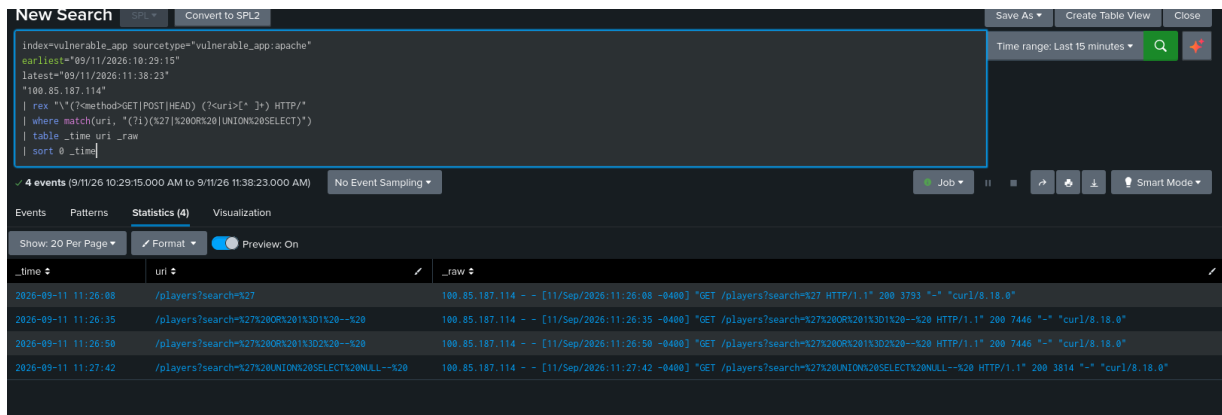


Figure 4.39 – Broad SQL Injection Indicator Detection

Evidence interpretation. The broader detection includes the initial quote probe together with encoded Boolean and UNION SELECT patterns. It demonstrates payload-level visibility for URI-based SQLi attempts.

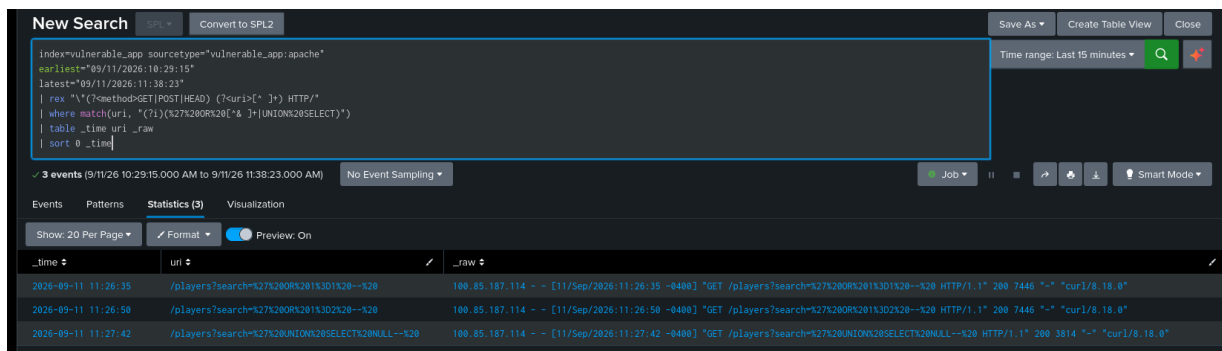


Figure 4.40 – Refined SQL Injection Detection Result

Evidence interpretation. The refined rule returns the stronger Boolean and UNION SELECT indicators. This improves specificity while retaining the evidence of structured SQLi testing.

4.5 Query Validation – Evidence of Firing Against Log Data

Table 4.4 – Original-Window Detection Validation

Detection	Original-window result	What the evidence supports
File Upload	5 POST /documents events	Upload attempts reached the endpoint; host evidence confirms two created artifacts.
XSS	3 POST /xss events	XSS endpoint activity was detected; exact body unavailable.
OS Command Injection	6 POST /diagnostics events	Diagnostics attack traffic was detected; exact body unavailable.
SQL Injection	Boolean and UNION SELECT probes recorded	SQLi-style URI values were observable.
Enumeration	VM3 high-volume query result	Volume, URI diversity, and 404 pattern support automated enumeration.

CROSS-QUERY VALIDATION EVIDENCE

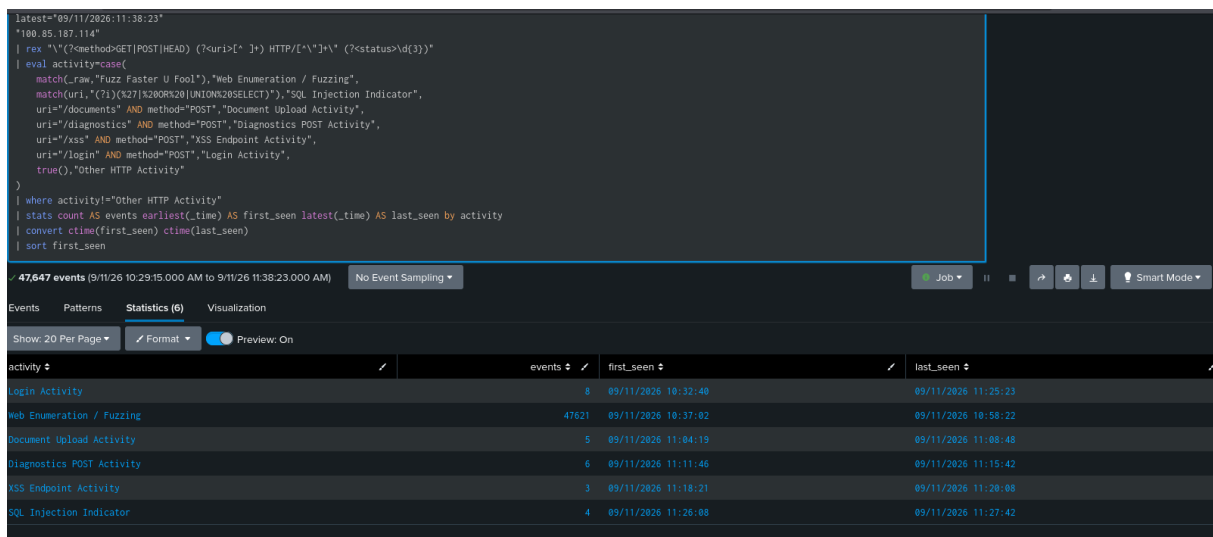


Figure 4.41 – Attack Activity Classification and Timing Summary

Evidence interpretation. This Splunk analytic classifies major observed activities and summarizes event counts with first/last-seen times. It provides a higher-level view of the attack phases for validation.

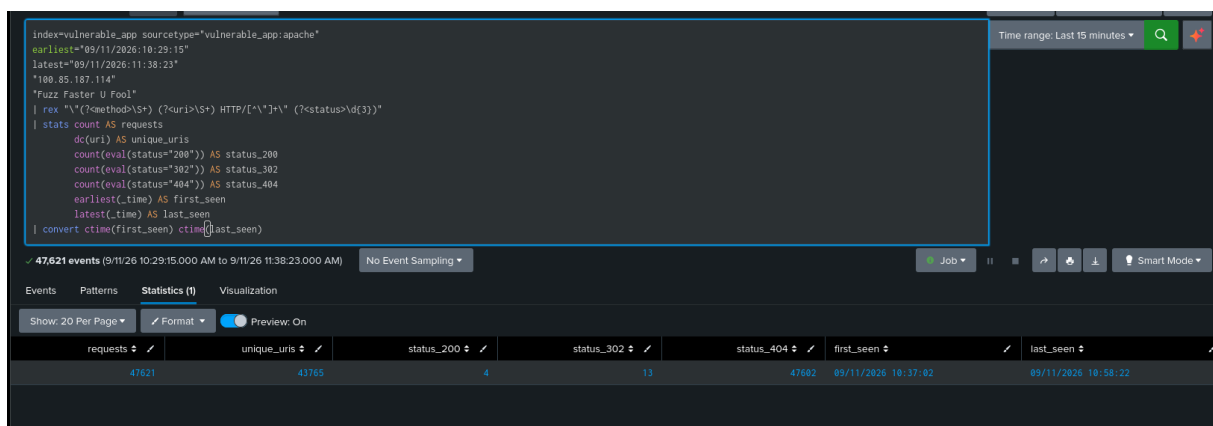


Figure 4.42 – FFUF / Fuzzing Behavioral Summary

Evidence interpretation. The aggregation records 47,720 requests, 43,776 unique URIs and 47,607 HTTP 404 responses from VM3. These metrics strongly support automated content enumeration.

Section 5: Mitigation & Re-Exploitation

STUDENT 4 — MOHAMMAD AIMAN ALI (ID 2023904141) / MITIGATION & RE-EXPLOITATION; RE-TEST EXECUTED WITH RED TEAM SUPPORT

5.1 Version Control Setup

5.1.1 Git Repository Structure

Git was initialized in /home/kali/vulnerable-app so that the vulnerable state and each security fix could be tracked independently. The repository contains the application source, templates, static assets, and supporting files, while the evidence shows a clean working tree at the initial baseline.

```
(kali@kali)-[~/vulnerable-app]
$ git log --oneline
58e0dae (HEAD -> master) Initial vulnerable application baseline

(kali@kali)-[~/vulnerable-app]
$ git status
On branch master
nothing to commit, working tree clean

(kali@kali)-[~/vulnerable-app]
$
```

Figure 5.1 – Git baseline: initial vulnerable application commit with a clean working tree.

5.1.2 Initial Commit Vulnerable Baseline

```
58e0dae Initial vulnerable application baseline
```

This baseline established a recoverable reference point before any security patch was applied. Subsequent commits isolate one remediation area at a time, which makes the change history auditable.

5.2 Code Remediation

5.2.1 Fix for Vulnerability 1 — File Upload

The vulnerable upload path saved submitted filenames without an effective file-type control. The patch introduced an extension allowlist (txt, pdf, png, jpg, jpeg), a maximum content length, and a server-side `allowed_file()` check before saving. Disallowed PHP-family uploads now return “File type is not allowed,” while a normal text upload continues to work.

```
(kali@kali)-[~/vulnerable-app]
$ nl -ba app.py | sed -n '10,30p'
10
11 app.secret_key = "urban-sports-lab-secret-key"
12
13 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
14 DATABASE = os.path.join(BASE_DIR, "database.db")
15 UPLOAD_FOLDER = os.path.join(BASE_DIR, "uploads")
16
17 ALLOWED_EXTENSIONS = {"txt", "pdf", "png", "jpg", "jpeg"}
18 app.config["MAX_CONTENT_LENGTH"] = 2 * 1024 * 1024
19
20
21 def allowed_file(filename):
22     return (
23         "." in filename
24         and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS
25     )
26
27
28 os.makedirs(UPLOAD_FOLDER, exist_ok=True)
29
30
```

Figure 5.2 – File-upload remediation configuration showing the allowlisted extensions, size limit, and `allowed_file()` helper.

```

(kali@kali)-[~/vulnerable-app]
$ nl -ba app.py | sed -n '305,335p'
305     connection.close()
306
307     return render_template("matches.html", matches=match_list)
308
309
310 @app.route("/documents", methods=["GET", "POST"])
311 def documents():
312     if "username" not in session:
313         return redirect(url_for("login"))
314
315     message = None
316     uploaded_file = None
317
318     if request.method == "POST":
319         uploaded_file = request.files.get("document")
320
321         if uploaded_file and uploaded_file.filename:
322             filename = secure_filename(uploaded_file.filename)
323
324             if filename and allowed_file(filename):
325                 save_path = os.path.join(UPLOAD_FOLDER, filename)
326                 uploaded_file.save(save_path)
327
328                 message = f"Document '{filename}' was uploaded successfully."
329             else:
330                 message = "File type is not allowed."
331
332     existing_files = []
333
334     for filename in os.listdir(UPLOAD_FOLDER):
335         existing_files.append(filename)

```

Figure 5.3 – Patched Documents route applying `allowed_file()` before saving an upload.

The screenshot shows a web application interface for uploading documents. At the top, a green message box displays the text "File type is not allowed." Below this, the "Upload Document" section has a subtitle "Add a document to the sports management system." In the center, there is a large light blue box with an upward-pointing arrow and the text "Select a document". Below this, a smaller instruction reads "Upload a team document, match report or player file." A "Browse..." button is followed by the text "No file selected." At the bottom of the section is a dark blue "Upload document" button.

Figure 5.4 – Post-fix application response rejecting a disallowed upload with "File type is not allowed."

5.2.2 Fix for Vulnerability 2 — OS Command Injection

The vulnerable Diagnostics flow allowed a host value to reach shell interpretation. The patch validates the submitted target with `ipaddress.ip_address()` and passes a list of arguments directly to `subprocess.check_output()` without invoking a shell. Invalid composite input such as `127.0.0.1; whoami` is rejected before piping executes.

```

@app.route("/diagnostics", methods=["GET", "POST"])
def diagnostics():
    if "username" not in session:
        return redirect(url_for("login"))

    result = None

    if request.method == "POST":
        target = request.form.get("target", "").strip()

        # Validate the target and execute ping without invoking a shell.
        try:
            validated_target = str(ipaddress.ip_address(target))

            result = subprocess.check_output(
                ["ping", "-c", "1", validated_target],
                stderr=subprocess.STDOUT,
                text=True,
                timeout=10,
            )
        except ValueError:
            result = "Invalid IP address."
        except subprocess.CalledProcessError as error:
            result = error.output
        except Exception as error:
            result = str(error)

    return render_template(
        "diagnostics.html",
        result=result,
    )

```

Figure 5.5 – Patched Diagnostics route validating an IP address and executing ping without shell=True.

The screenshot shows a web application interface with three main sections:

- Connectivity Check:** A section with the subtext "Verify connectivity to a remote service or host." It contains a text input field with the value "127.0.0.1; whoami" and a button labeled "Run connectivity check".
- Service Overview:** A section titled "Current application services." containing a table:

Service	Status
Web Application Primary application service	Online
Database Application data storage	Online
Web Server HTTP service	Online
- Diagnostic Result:** A section titled "Output from the connectivity check." showing a dark box with the text "Invalid IP address."

Figure 5.6 – Application result rejecting manipulated Diagnostics input as an invalid IP address.

5.2.3 Fix for Vulnerability 3 — Stored XSS

The vulnerable template rendered announcement content using the Jinja safe filter, which bypassed normal escaping. The fix removed `|safe` so untrusted announcement text is escaped by the template engine. The controlled script string is therefore displayed literally rather than interpreted as HTML/JavaScript.



Figure 5.7 – Vulnerable XSS template showing message|safe before remediation.

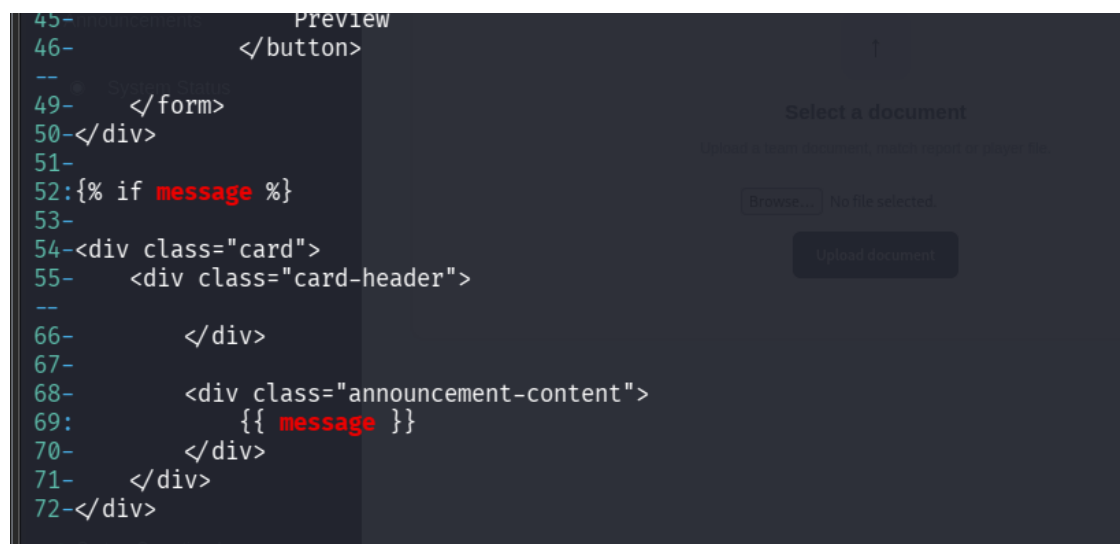


Figure 5.8 – Patched XSS template rendering message without the unsafe safe filter.

Community Announcements

Publish updates and messages for players and team members.

Content Management

Create Announcement

Share an important update with the sports community.

```
<script>alert(1)</script>
```

Announcement

Publish Announcement

Preview

Latest Announcement

Recently published community update.

Urban Sports Management Just now

```
<script>alert(1)</script>
```

Recent Updates

Community communication overview.

Matchday Update Next fixtures are scheduled for this week.

Player Registration Registration remains open for new players.

Facility Notice Amman Sports City maintenance is scheduled this weekend.

Figure 5.9 – Post-fix Community Announcements page displaying script content as text rather than executing it.

5.2.4 Fix for Vulnerability 4 — SQL Injection

The vulnerable login query interpolated username and password directly into SQL. The remediation replaced string construction with parameterized placeholders and bound parameters. The Players search was likewise changed to LIKE ? with a bound search pattern. The previously successful login bypass now returns the normal authentication failure message.

```
(kali@kali)-[~/vulnerable-app]
$ nl -ba app.py | sed -n '197,230p'
197     "SELECT id, username, role FROM users "
198     f"WHERE username = '{username}' "
199     f"AND password = '{password}'"
200 )
201
202     try:
203         user = connection.execute(query).fetchone()
204     except sqlite3.Error as database_error:
205         user = None
206         error = f"Unable to complete sign-in: {database_error}"
207
208     connection.close()
209
210     if user:
211         session["username"] = user["username"]
212         session["role"] = user["role"]
213         return redirect(url_for("dashboard"))
214
215     if error is None:
216         error = "The username or password could not be verified."
217
218     return render_template("login.html", error=error)
219
220
221 @app.route("/logout")
222 def logout():
223     session.clear()
224     return redirect(url_for("login"))
225
226
227 @app.route("/teams")
228 def teams():
229     if "username" not in session:
230         return redirect(url_for("login"))
```

Figure 5.10 – Pre-fix login code constructing SQL using direct string interpolation.

```

(kali@kali) ~/vulnerable-app
$ nl -ba app.py | sed -n '190,220p'
190 username = request.form.get("username", "")
191 password = request.form.get("password", "")
192
193 connection = get_db()
194
195 # Intentionally vulnerable for the controlled cybersecurity lab.
196 query = """
197     SELECT id, username, role FROM users
198     WHERE username = ?
199     AND password = ?
200 """
201
202 try:
203     user = connection.execute(
204         query,
205         (username, password),
206     ).fetchone()
207 except sqlite3.Error as database_error:
208     user = None
209     error = f"Unable to complete sign-in: {database_error}"
210
211 connection.close()
212
213 if user:
214     session["username"] = user["username"]
215     session["role"] = user["role"]
216     return redirect(url_for("dashboard"))
217
218 if error is None:
219     error = "The username or password could not be verified."
220

```

Figure 5.11 – Post-fix login code using parameterized SQL with bound username and password values.

```

242
243
244 @app.route("/players")
245 def players():
246     if "username" not in session:
247         return redirect(url_for("login"))
248
249     search = request.args.get("search", "")
250
251     connection = get_db()
252
253     # Use parameterized queries for player search.
254     search_pattern = f"%{search}%"
255
256     query = """
257         SELECT * FROM players
258         WHERE name LIKE ?
259             OR position LIKE ?
260             OR team LIKE ?
261         ORDER BY name
262     """
263
264     try:
265         player_list = connection.execute(
266             query,
267             (search_pattern, search_pattern, search_pattern),
268         ).fetchall()
269         database_error = None
270     except sqlite3.Error as error:
271         player_list = []
272         database_error = str(error)
273
274     connection.close()
275

```

Figure 5.12 – Post-fix Players search using a parameterized LIKE query.

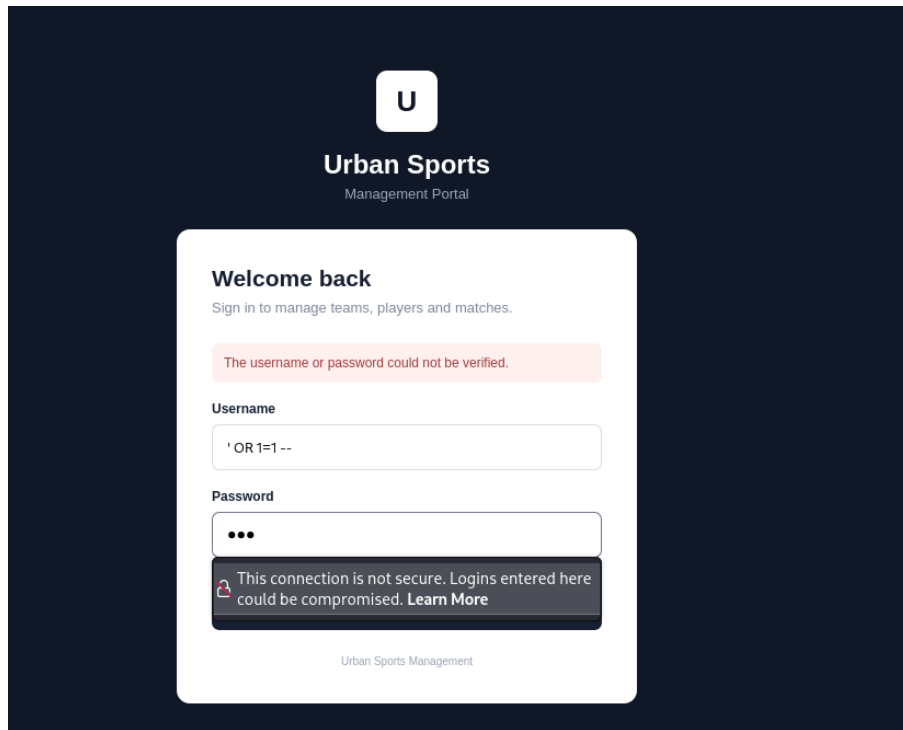


Figure 5.13 – Post-fix login attempt rejected rather than creating an Administrator session.

5.3 Patched Application Deployment

After the fixes were committed, the application stack was returned to service on VM1. The final service verification shows tailscaled, apache2, and vulnerable-app in the active state. A benign connectivity check against 127.0.0.1 also returned the expected ping result, demonstrating that the security change preserved intended Diagnostics functionality for valid input.

```

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo systemctl is-active tailscaled
[sudo] password for kali:
active

(kali@kali)-[~]
$ sudo systemctl is-active apache2
active

(kali@kali)-[~]
$ sudo systemctl is-active vulnerable-app
active

(kali@kali)-[~]
$ ls -la ~/vulnerable-app
total 44
drwxrwxr-x 7 kali kali 4096 Sep  5 17:03 .
drwx----- 20 kali kali 4096 Sep 11 08:47 ..
-rw-rw-r-- 1 kali kali 4004 Sep  5 16:53 app.py
-rw-r--r-- 1 kali kali 12288 Sep  5 16:56 database.db
drwxrwxr-x 2 kali kali 4096 Sep  5 17:03 __pycache__
drwxrwxr-x 2 kali kali 4096 Sep  5 16:56 static
drwxrwxr-x 2 kali kali 4096 Sep  5 18:34 templates
drwxrwxr-x 2 kali kali 4096 Sep  7 13:34 uploads
drwxrwxr-x 5 kali kali 4096 Sep  5 16:51 venv
  
```

Figure 5.14 – Post-remediation service state on VM1 showing the VPN, web server, and application service active.

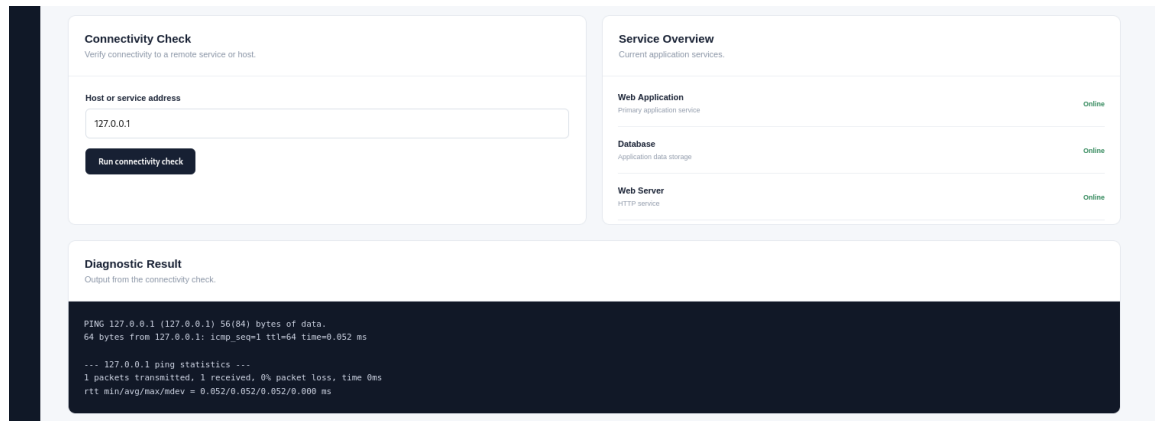


Figure 5.15 – Valid 127.0.0.1 connectivity check continues to work after the command-injection fix.

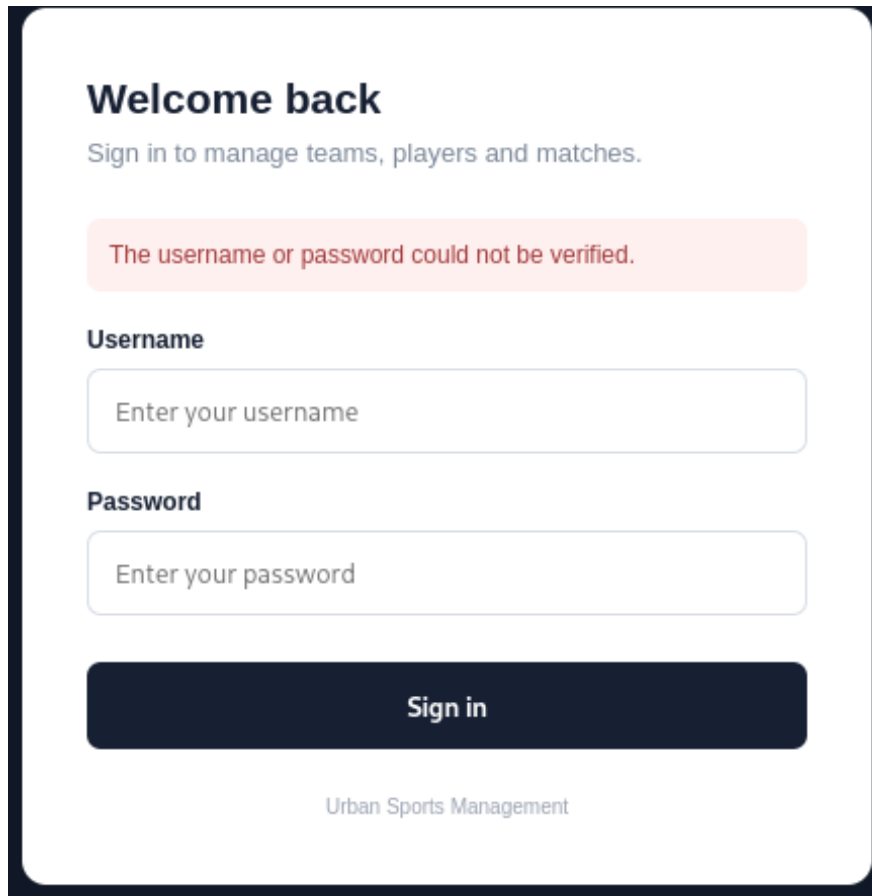
5.4 Re-Exploitation Testing

5.4.1 Re-test Results Against PT Report Findings

Student 2 repeated the original or equivalent proof-of-concept inputs against the patched application. Reverse-shell execution was not repeated; command execution was safely re-tested using a non-destructive id payload. The four retests produced results consistent with remediation of the exercised attack paths.

Table 5.1 – Application Re-Test Summary

Finding	Original impact	Post-patch re-test	Status
SQL Injection — Login	Administrator authentication bypass	Same Boolean-style input rejected; no admin session obtained.	FIXED
OS Command Injection / RCE	Arbitrary command execution / reverse shell	Manipulated host input rejected; no uid/gid output.	FIXED
Unrestricted File Upload	PHP-family file accepted	shell.php5 rejected as disallowed type.	FIXED
Stored XSS	Persistent JavaScript execution	Payload rendered as literal text; no alert execution.	FIXED



Welcome back

Sign in to manage teams, players and matches.

The username or password could not be verified.

Username

Enter your username

Password

Enter your password

Sign in

Urban Sports Management

Figure 5.16 – SQL-injection authentication-bypass re-test remains on the sign-in page with the normal failure message.



Diagnostic Result

Output from the connectivity check.

Invalid IP address.

Figure 5.17 – Command-injection re-test returns “Invalid IP address” and no command output.

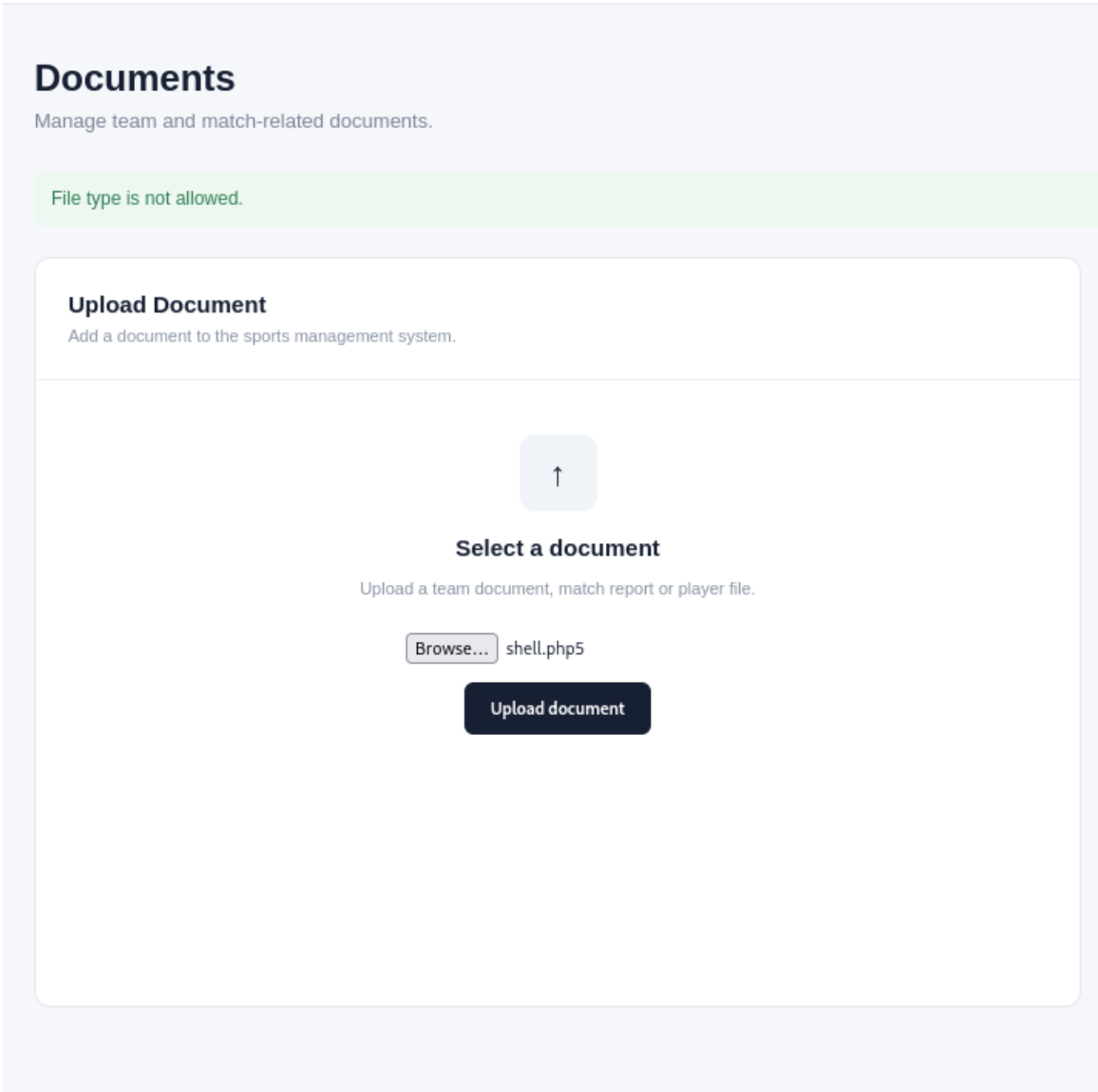


Figure 5.18 – PHP-family upload rejected after remediation.



Figure 5.19 – Stored-XSS payload rendered as text rather than executed.

5.4.2 Confirmation of Patch Effectiveness

The re-test results demonstrate that the four previously confirmed impacts were not reproduced with the tested proof-of-concept inputs after remediation. This is strong evidence for the exercised fixes, but it is intentionally narrower than claiming the application is free from every possible bypass or unrelated weakness. Normal valid-input checks also show that the intended upload and Diagnostics functions remained operational.

5.5 Detection Query Verification (By Student 3)

5.5.1 Re-run Exploit Attempt on Patched App

After the patched application was deployed, test attempts were repeated from VM3 during the post-patch window 16:59:09–17:26:51 EDT on 11 September 2026. Splunk returned 73 Apache events from 100.85.187.114 in that interval. These events include navigation and supporting requests; the count is not a count of successful exploits.

Table 5.2 – Observed Re-Test Sequence

Observed time EDT	Re-test activity	Detection matches
16:59:15	SQL injection probe	1 URI-level SQLi match
17:04:43–17:05:37	OS command injection test	2 Diagnostics POST matches
17:20:22–17:23:21	File upload tests	4 Documents POST matches
17:24:31–17:25:25	XSS tests	6 XSS POST matches

POST-PATCH SIEM SCOPE

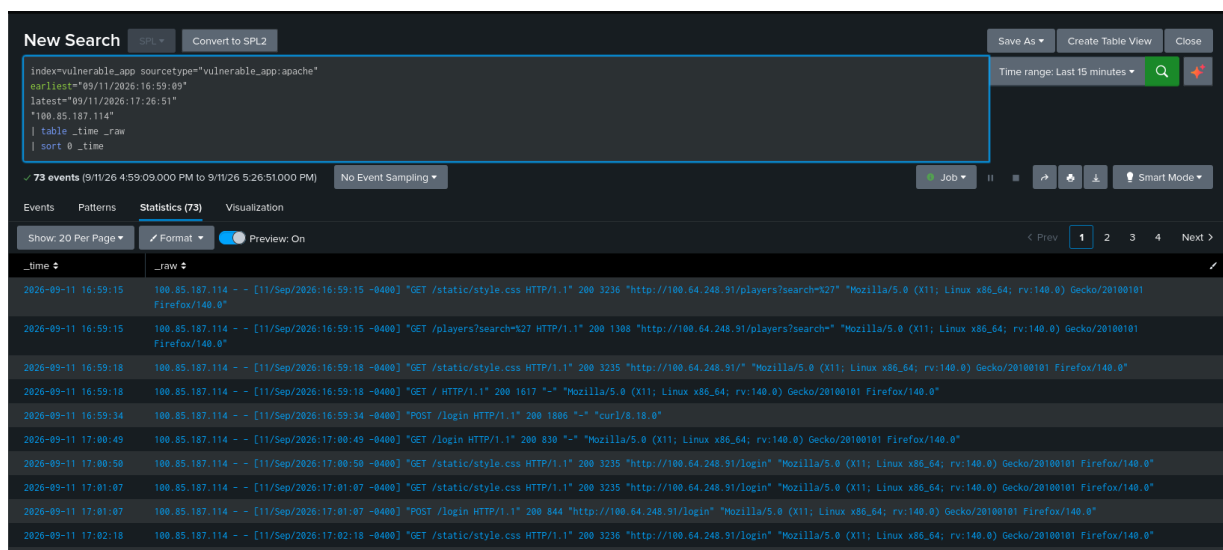


Figure 5.20 – Post-Patch SIEM Re-Test Scope

Evidence interpretation. The post-patch search is bounded to 16:59:09–17:26:51 EDT and the attacker source 100.85.187.114. It establishes the log population used for detection verification after remediation.

5.5.2 Confirmation That Student 3 Queries Fire on Re-test Logs

All four Student 3 detections continued to fire on the post-patch re-test traffic. This demonstrates continued observability even though the application-level exploit impacts were blocked. For the POST endpoints, the SIEM sees endpoint activity rather than the exact POST body; the SQL test provides direct URI-level visibility of the quote probe.

Table 5.3 – Post-Patch Detection Verification

Detection	Matches	What the result verifies
File Upload	4	Upload-endpoint re-test activity was detected.
XSS	6	XSS-endpoint re-test activity was detected.
OS Command Injection	2	Diagnostics re-test activity was detected.
SQL Injection	1	Decoded quote probe in /players?search= was detected.

POST-PATCH DETECTION EVIDENCE

New Search SPL Convert to SPL2 Save As Create Table View Close

Time range: Last 15 minutes Q +

```
index=vulnerable_app sourcetype=vulnerable_app:apache
earliest=09/11/2026:16:59:09
latest=09/11/2026:17:26:51
| rex "(?<src_ip>S+)"
| rex "\.(?<method>S+) (?<uri>S+) HTTP/[^\"]+\" (?<status>S+)"
| where method=POST AND like(uri, "/documents%")
| table _time src_ip method uri status
| sort 0 _time
```

✓ 4 events (9/11/26 4:59:09.000 PM to 9/11/26 5:26:51.000 PM) No Event Sampling Job II ≡ ↶ ↷ ⬇ ⬆ Smart Mode

Events Patterns Statistics (4) Visualization

Show: 20 Per Page Format Preview: On

_time	src_ip	method	uri	status
2026-09-11 17:20:22	100.85.187.114	POST	/documents	200
2026-09-11 17:20:33	100.85.187.114	POST	/documents	200
2026-09-11 17:22:09	100.85.187.114	POST	/documents	200
2026-09-11 17:23:21	100.85.187.114	POST	/documents	200

Figure 5.21 – Post-Patch File Upload Detection Fired

Evidence interpretation. The File Upload rule detects the re-test POST requests to /documents. A detection match proves the attempt was observed, while the patched application separately rejected the disallowed PHP-family file.

New Search SPL Convert to SPL2 Save As Create Table View Close

Time range: Last 15 minutes Q +

```
index=vulnerable_app sourcetype=vulnerable_app:apache
earliest=09/11/2026:16:59:09
latest=09/11/2026:17:26:51
| rex "(?<src_ip>S+)"
| rex "\.(?<method>S+) (?<uri>S+) HTTP/[^\"]+\" (?<status>S+)"
| eval decoded_uri=urldecode(uri)
| where src_ip=100.85.187.114
AND (
  method=POST AND like(uri, "/xss%")
OR match(decoded_uri, "(?i)<script[\\s]*javascript:|onerror\\s*=|<svg|<img)")
| table _time src_ip method uri status decoded_uri
| sort 0 _time
```

✓ 6 events (9/11/26 4:59:09.000 PM to 9/11/26 5:26:51.000 PM) No Event Sampling Job II ≡ ↶ ↷ ⬇ ⬆ Smart Mode

Events Patterns Statistics (6) Visualization

Show: 20 Per Page Format Preview: On

_time	src_ip	method	uri	status	decoded_uri
2026-09-11 17:24:31	100.85.187.114	POST	/xss	200	/xss
2026-09-11 17:24:34	100.85.187.114	POST	/xss	200	/xss
2026-09-11 17:24:36	100.85.187.114	POST	/xss	200	/xss
2026-09-11 17:24:37	100.85.187.114	POST	/xss	200	/xss
2026-09-11 17:24:40	100.85.187.114	POST	/xss	200	/xss
2026-09-11 17:25:25	100.85.187.114	POST	/xss	200	/xss

Figure 5.22 – Post-Patch XSS Detection Fired

Evidence interpretation. The XSS rule detects the re-test POST requests to /xss. The application-level re-test shows the script input was rendered as text rather than executed.

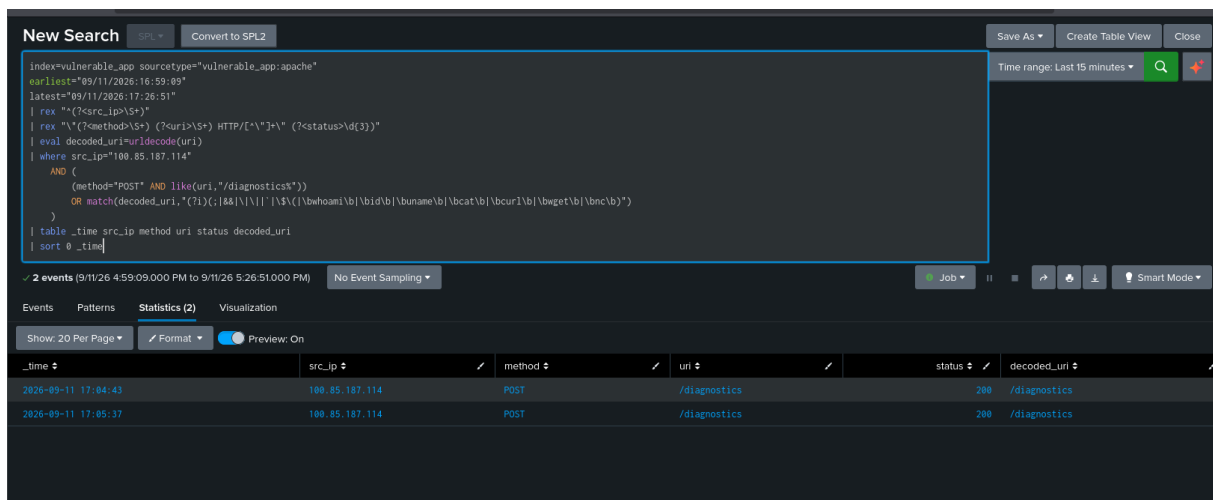


Figure 5.23 – Post-Patch OS Command Injection Detection Fired

Evidence interpretation. The Diagnostics rule detects the re-test requests to /diagnostics. The patched application rejected the manipulated target as an invalid IP address and produced no command output.

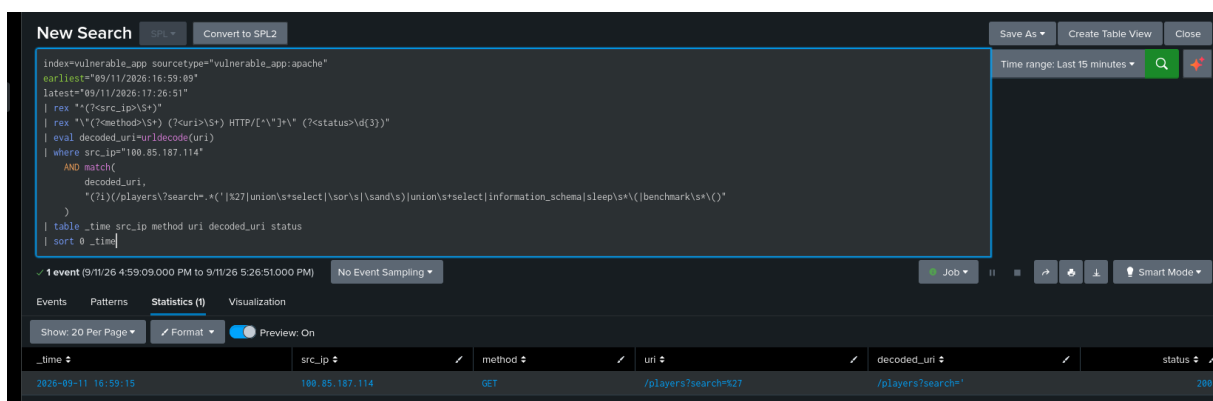


Figure 5.24 – Post-Patch SQL Injection Detection Fired

Evidence interpretation. The SQLi rule detects the encoded search probe in /players?search=. The match demonstrates continued SIEM visibility after remediation; it does not claim database compromise.

5.6 Git Commit History Summary

Table 5.4 – Security-Focused Git History

Commit	Message	Security change
58e0dae	Initial vulnerable application baseline	Pre-fix reference state
e364a50	Fix OS command injection in diagnostics	IP validation + shell-free subprocess
52b5544	Fix SQL injection using parameterized queries	Bound SQL parameters for login and Players search
5f35d89	Fix unrestricted file upload vulnerability	Allowlist / size validation before save
802c7b4	Fix XSS by enabling template output escaping	Removed unsafe template rendering

```
(kali@kali)-[~/vulnerable-app]
$ git add templates/xss.html

(kali@kali)-[~/vulnerable-app]
$ git commit -m "Fix XSS by enabling template output escaping"
[master 802c7b4] Fix XSS by enabling template output escaping
1 file changed, 1 insertion(+), 1 deletion(-)

(kali@kali)-[~/vulnerable-app]
$ git log --oneline -5
802c7b4 (HEAD -> master) Fix XSS by enabling template output escaping
5f35d89 Fix unrestricted file upload vulnerability
52b5544 Fix SQL injection using parameterized queries
e364a50 Fix OS command injection in diagnostics
58e0dae Initial vulnerable application baseline
```

Figure 5.25 – Final Git log after the XSS commit showing the complete security remediation sequence.

Section 6: Conclusion

6.1 Project Summary

The project completed the intended end-to-end lifecycle: the team built a deliberately vulnerable application, connected distributed lab systems through Tailscale, centralized logs in Splunk, performed controlled exploitation, reconstructed and contained the incident, patched the vulnerable code, and re-tested both prevention and detection. The final state is not merely “patched”; it is supported by evidence from application responses, host inspection, Git history, and SIEM validation.

6.2 Key Findings Across All Roles

- The offensive assessment demonstrated a practical chain from login SQL injection to Administrator access and then OS command injection / RCE with a controlled reverse shell.
- Stored XSS and unrestricted file upload were independently validated; the upload weakness allowed a PHP-family file to be stored, but direct PHP execution was not demonstrated.
- Splunk reconstructed the attack primarily from Apache access logs, while Zsh history provided only pre-attack baseline visibility and did not capture commands executed by the web process.
- Host triage correlated two attack-window upload artifacts with HTTP timestamps, recorded metadata and hashes, removed the artifacts, and found no obvious persistence in the inspected cron/systemd/autostart checks.
- All four code fixes blocked the tested proof-of-concept impacts, and all four defensive detections continued to match the post-patch re-test traffic.

6.3 Lessons Learned

The strongest lesson was the need to separate evidence of an attack attempt from evidence of successful exploitation. Access-log matches and HTTP 200 responses show that traffic reached the application, but successful impact requires corroboration from the application or host. The project also showed why interactive shell history is insufficient as process-execution telemetry: the web process can execute commands without writing them to the user’s Zsh history.

From an engineering perspective, small code-level design decisions produced major security differences: parameterized queries removed SQL interpretation of user input, input validation plus shell-free subprocess execution removed shell metacharacter interpretation, an upload allowlist reduced executable-file storage, and ordinary template escaping prevented browser execution of stored script content.

6.4 Future Improvements

- Add application-level security logging for authentication failures, upload acceptance/rejection, input-validation failures, and privileged actions.
- Add host process-execution telemetry (for example audit or endpoint telemetry) so web-triggered command execution can be reconstructed directly.
- Capture structured reverse-proxy or application request metadata for security-relevant parameters while avoiding unnecessary sensitive-data logging.
- Automate the Splunk detections as saved searches/alerts with thresholds and enrichment for the three lab IPs.
- Expand re-testing with negative and boundary cases, filename/content mismatches, encoded XSS variants, and parameterized-query regression tests.
- Record VM CPU/RAM/disk allocations and all installation commands in the architecture evidence package for more complete reproducibility.

References

The final report is grounded in the project evidence supplied by the team. No external source was used to replace or override the observed lab results.

1. Hadi Bakeer. Penetration Testing Report — Urban Sports Management Web Application. Controlled project write-up, September 2026.
2. Mohammad Aiman Ali. Incident Response & Detection — Student 3. Individual technical report, expanded SIEM setup/evidence package (student3(2)), and Splunk screenshots, 12 September 2026.
3. Urban Sports Management Penetration Testing Retesting Report. Remediation verification phase, 11 September 2026.
4. Yousef Moayad. VM1 application / architecture evidence package (service state, code, application views, and directory structure).
5. Mohammad Aiman Ali (Student 4). Remediation evidence package: Git baseline, code modifications, fix commits, functional checks, and re-exploitation verification.
6. Team screenshots: Tailscale peer status and VM1 active-service verification.